TOKYO METRO ROUTE FINDER USING DIJKSTRA ALGORITHM

YU YIFAN

UNIVERSITI KEBANGSAAN MALAYSIA

TOKYO METRO ROUTE FINDER USING DIJKSTRA ALGORITHM

YU YIFAN

REPORT SUBMITTED TO FULFILL PART OF THE REQUIREMENTS FOR OBTAINING THE DEGREE OF BACHELOR OF COMPUTER SCIENCE WITH HONOURS

FACULTY SCIENCE AND TECHNOLOGY
UNIVERSITI KEBANGSAAN MALAYSIA
BANGI

2026

DECLARATION

I hereby declare that the work in this thesis is my own except for quotations and summaries, which have been duly acknowledged

23 June 2026

YU YIFAN

A191702

ACKNOWLEDGEMENT

I would like to sincerely thank my supervisor, Ts. Dr. Mohd Nor Akmal Bin Khalid, for his continuous valuable guidance in the process of completing my thesis. It is my honor to study under his guidance. Thanks to his patience, support, tolerance, and encouragement, I have the confidence and courage to move forward. Special thanks are also extended to my examiner, Assoc. Prof. Dr. Zalinda Binti Othman, for her constructive feedback, valuable insights, and time spent reviewing this work.

I would also like to give special thanks to the professors at Fakulti Teknologi dan Sains Maklumat, Universiti Kebangsaan Malaysia for their guidance and support over the past few years. The education during this period not only enabled me to master professional knowledge, but also greatly broadened my vision, laying a solid foundation for my academic development and career planning.

At the same time, I would also like to thank my parents, family and friends. Their love and care are the strongest backing for me when I meet difficulties and make me a better person.

A very special thank you goes to my girlfriend, ZHU JINGWEN, for her unwavering emotional support, patience, and belief in me. Her presence helped me stay grounded and motivated throughout this journey, especially during times of stress and uncertainty. You may not have written a single line of code, but your love has been in every line I typed. 君の笑顔が、僕のプログラムの中で一番美しいコードでした。ありがとう、愛しています。谢谢你，我爱你，未来也请多多指教。

Finally, I would like to thank those authors whose papers and words gave me the inspiration to write this paper.

Abstract

Urban metro systems in large metropolitan areas such as Tokyo are characterized by dense networks, complex transfer structures, and high passenger demand. Efficient route planning within such environments is therefore not only a practical necessity for daily commuters, but also an important research topic in the fields of algorithm design and transportation systems.

This project proposes a Tokyo Metro Route Finder that applies Dijkstra's shortest path algorithm to identify optimal travel routes between selected stations. Using real-world metro station data, the system models the metro network as a weighted graph, where stations are represented as nodes and travel times between stations are treated as edge weights. Based on user-defined origin and destination stations, the algorithm computes the shortest path and corresponding total travel time.

To enhance usability and educational value, the algorithm is implemented using Python and integrated into a lightweight web-based prototype developed with the Flask framework. The system allows users to interactively query routes and visually inspect the computed results, thereby improving transparency compared to commercial navigation platforms. Overall, this project demonstrates how classical graph algorithms can be effectively applied to real-world transportation problems, while also serving as a practical learning tool for algorithm visualization and system development.

Abstrak

Sistem pengangkutan metro di bandar metropolitan besar seperti Tokyo mempunyai rangkaian yang padat, struktur pertukaran yang kompleks serta permintaan penumpang yang tinggi. Oleh itu, perancangan laluan yang cekap bukan sahaja penting untuk kegunaan harian penumpang, malah menjadi satu topik kajian yang signifikan dalam bidang reka bentuk algoritma dan sistem pengangkutan.

Projek ini membangunkan satu sistem pencarian laluan Tokyo Metro yang menggunakan algoritma laluan terpendek Dijkstra bagi menentukan laluan perjalanan yang optimum antara dua stesen. Rangkaian metro dimodelkan sebagai graf berwajaran, di mana stesen diwakili sebagai nod dan masa perjalanan antara stesen digunakan sebagai pemberat sisi. Berdasarkan stesen asal dan destinasi yang dipilih oleh pengguna, sistem akan mengira laluan terpendek serta jumlah masa perjalanan.

Bagi meningkatkan kebolehgunaan dan nilai pembelajaran, algoritma ini dilaksanakan menggunakan Python dan diintegrasikan ke dalam prototaip berasaskan web menggunakan rangka kerja Flask. Pendekatan ini membolehkan pengguna berinteraksi secara langsung dengan sistem serta memahami hasil pengiraan algoritma dengan lebih jelas. Secara keseluruhannya, projek ini menunjukkan aplikasi praktikal algoritma graf klasik dalam konteks dunia sebenar, khususnya bagi sistem pengangkutan bandar.

要旨

東京のような大都市圏における地下鉄システムは、路線網が非常に密集しており、乗換構造も複雑である。そのため、効率的な経路探索は日常的な利用者にとって重要であるだけでなく、アルゴリズム設計や交通システム研究においても意義のある課題となっている。

本研究では、ダイクストラ法を用いた東京メトロ経路探索システムを提案する。地下鉄ネットワークは重み付きグラフとしてモデル化され、各駅をノード、駅間の移動時間をエッジの重みとして定義する。利用者が出発駅と到着駅を入力すると、アルゴリズムにより最短経路および総移動時間が算出される。

さらに、本システムは Python を用いて実装され、Flask フレームワークを活用した軽量な Web プロトタイプとして構築されている。これにより、商用ナビゲーションサービスでは確認できない経路計算の過程を可視化し、教育的価値と透明性を向上させている。本研究は、古典的なグラフアルゴリズムが実世界の都市交通問題にどのように応用できるかを示す実践的な事例である。

TABLE OF CONTENTS

LIST OF ILLUSTRATIONS

LIST OF TABLES

CHAPTER I

PROJECT PLANNING

1.1 INTRODUCTION

The Tokyo Metro system stands as one of the most complex and densely utilized urban rail networks globally. Its intricate web of lines and stations presents a significant challenge for passengers seeking to navigate the city efficiently. While existing official and third-party journey planners provide route suggestions, they often prioritize shortest distance or fare cost. There is a discernible gap in accessible, educational prototypes that specifically model and compute the shortest travel time, incorporating realistic transfer penalties. Furthermore, many academic demonstrations of pathfinding algorithms utilize abstract or simplified graphs, lacking implementation with real-world station data and approximate real-time travel constraints.

This Final Year Project aims to address this gap by developing a functional prototype of a route inquiry system for the Tokyo Metro. The core of this system will be based on graph theory principles and the Dijkstra shortest-path algorithm. The project encompasses algorithm design, data processing, and front-end implementation, aligning perfectly with the learning outcomes of the Computer Science program, which emphasizes algorithm application, system development, and the use of real-world data. The prototype will feature both a Command-Line Interface (CLI) for backend testing and a lightweight web front-end for demonstration purposes. This architecture is intentionally designed to allow for future enhancements, such as the integration of the A* algorithm, General Transit Feed Specification (GTFS) data, and interactive map visualizations.

1.2 PROBLEM STATEMENT

The central challenge this project addresses is the lack of an accessible, educational, and algorithmically transparent tool for computing the shortest time-based path across the Tokyo Metro network, which accurately accounts for the operational reality of transfer penalties. Existing public-facing applications, while practical, do not expose the underlying computational models or allow for experimentation with parameters like transfer time. Conversely, typical academic projects that implement Dijkstra's algorithm often lack the nuance of real-world data, failing to model complex network features like multi-line interchange stations [5, 8]. This project specifically tackles the problem of transforming the physical metro network into a computationally viable model and executing an efficient pathfinding algorithm upon it. The key research questions are:

Firstly, how can the Tokyo Metro network—with its stations, lines, and interconnections—be accurately abstracted into a weighted graph data structure where nodes represent stations and edges represent travel time, using either publicly available or realistically synthesized data?

Secondly, how can Dijkstra's classic algorithm be implemented and optimized (e.g., using a min-heap priority queue) to efficiently navigate this potentially large graph to find the path of least total travel time [1, 3]? Thirdly, and most critically, how can the significant time cost associated with passenger transfers between different lines at interchange stations be effectively incorporated into the graph model and the algorithm's logic? Neglecting this factor produces unrealistic routes, as a path with three short trains but two long transfers may be less optimal than a slightly longer single-train journey. Finally, how can this computational backend be paired with an intuitive, multi-faceted user interface that not only demonstrates the system's capability to end-users via a web front-end but also provides a testing and debugging interface for developers via a command-line tool?

1.3 PROPOSED SOLUTIONS

To comprehensively address the problems outlined, a multi-faceted solution is proposed, structured around several core modules. The first pillar involves Data Construction and Processing. The project will build a digital representation of the Tokyo Metro network. If available, official or crowd-sourced GTFS data will be used; otherwise, a representative dataset will be manually curated from public sources [4, 10]. This data will be structured into CSV files: a stations.csv file containing unique identifiers, names in English and Japanese, and geographical coordinates, and an edges.csv file defining the connections between stations, including travel time and line affiliation. A crucial aspect of this modeling will be the explicit representation of transfers, likely implemented as special edges with a configurable time penalty, connecting platforms within the same station complex.

The second pillar is the Core Algorithm Implementation. A Python-based implementation of Dijkstra's algorithm will be developed, utilizing a min-heap (from the heapq module) to ensure optimal computational efficiency [1, 3]. The algorithm will be designed to operate on the constructed graph, where the weight of an edge is its travel time. The innovation lies in seamlessly integrating the transfer penalty; when the algorithm's path transitions from one metro line to another at a station, the predefined penalty value will be added to the cumulative travel time. This forces the algorithm to prioritize routes with fewer or more convenient transfers, mirroring real-world passenger preferences.

The third pillar focuses on the System Interface and Presentation Layer. A dual-interface approach will be adopted to cater to different user needs. A Command-Line Interface (CLI) will be built for rapid testing and validation of the core algorithm, allowing technical users to input station IDs and receive immediate path results. Simultaneously, a lightweight web application will be developed using the Flask

micro-framework. This front-end will feature a simple form for selecting origin and destination stations and will dynamically display the computed route, step-by-step instructions, total time, and number of transfers. Furthermore, a RESTful JSON API endpoint (e.g., /api/route) will be exposed, decoupling the backend logic from the front-end presentation and enabling future integration with advanced interactive maps using libraries like Leaflet.js.

Finally, a rigorous Testing and Evaluation Framework will be established. This involves creating a suite of test cases to validate algorithmic correctness and conducting a parametric analysis to study the impact of different transfer penalty values on the resulting optimal paths, thereby determining a configuration that best approximates real-world conditions.

1.4 OBJECTIVE

The project's objectives are defined using the SMART criteria to ensure clarity and achievability. The primary goal is Specific: to deliver a fully functional prototype of the Tokyo Metro Route Finder, comprising a data pipeline, a core pathfinding engine with transfer logic, a CLI, and a web interface. The outcomes are Measurable; the system must successfully process a network of at least 100 stations, return query results for any station pair within one second under local testing conditions, and provide documented analysis for at least five distinct test scenarios. The project is Achievable as it leverages a well-understood technology stack (Python, Flask) and a foundational algorithm, all within the scope of a final-year computer science curriculum and the 14-week semester timeline. It is highly Relevant, fulfilling the program's emphasis on applying theoretical algorithms to practical, data-rich problems and developing a complete software system.

Finally, it is Timed, with key deliverables synchronized with the academic calendar: Project Planning (D1) in Week 3, Literature Review (D2) in Week 5, Requirements Analysis (D3) in Week 8, Design Specification (D4) in Week 12, and the Final Proposal Report (D5) and Pra-KID presentation in Week 14.

1.5 SCOPE

The boundaries of this project are carefully defined to ensure focus and timely completion. The work in-scope includes: the creation of a representative Tokyo Metro dataset in CSV format; the full implementation of Dijkstra's algorithm with configurable transfer penalties; the development of a functional CLI and a minimal, functional Flask web application; the execution of a defined set of functional and parametric tests; and the production of all required academic documentation (D1-D5). Conversely, several advanced features are explicitly out-of-scope to manage complexity. These include the integration of real-time data or dynamic scheduling; the development of a production-grade, high-availability deployment; and the implementation of advanced user interface features such as a fully interactive map visualization or a native mobile application. These are identified as valuable avenues for future work but are beyond the core demonstrative aims of this prototype. A basic REST API is included for modularity, while advanced map visualization is out-of-scope.

1.6 RESTRICTIONS

Several constraints and potential risks are acknowledged. Data Availability and Accuracy present a significant challenge; the project's realism is contingent on the quality of the source data, and if official data is unavailable, synthesized data may limit real-world applicability. The fixed Time Constraints of a single academic semester necessitate strict scope management, with a fallback plan to prioritize core algorithm and CLI functionality should delays occur. Technical Limitations are also considered; the use of CSV files and a simple Flask server is suitable for a prototype but is not designed for concurrent multi-user access. The model itself has an Accuracy Limitation, as travel times and transfer penalties are estimates, meaning the system's output is a heuristic guide rather than a real-time navigation tool. Finally, the project's direction remains subject to Supervisor and Examiner Feedback, and the plan may be adaptively refined based on their guidance.

1.7 METHODOLOGY

The Incremental Development model has been selected as the primary software development methodology for this project [9]. This approach is highly suitable due to the project's modular architecture, allowing it to be built in a series of distinct, verifiable increments: 1) Data Modeling, 2) Algorithm Core, 3) CLI, 4) Web Front-end, and 5) Testing & Documentation. Each increment will produce a demonstrable artifact, facilitating early and continuous feedback from the project supervisor during scheduled meetings. This iterative process aligns perfectly with the phased delivery of deliverables (D1-D5), effectively mitigating risk by validating core components before layering on complexity. The chosen Technology Stack is pragmatic and fit-for-purpose: Python 3.x will form the backbone for data processing and the algorithm, leveraging libraries like csv and heapq. The Flask framework will be used for its simplicity and flexibility in building the web layer. Data will be stored in flat CSV files for portability, and the entire project will be managed using Git for version control, ensuring a organized and traceable development process.

1.8 IMPLEMENTATION SCHEDULE

A detailed 14-week implementation plan has been formulated, structured around a Work Breakdown Structure (WBS). The project will commence with Project Initiation and Literature Review (Weeks 1-2), involving the initial supervisor meeting and foundational research for D2. This will be immediately followed by the critical Data Collection and Modeling phase (Weeks 2-4), running in parallel with the start of Core Algorithm Implementation (Weeks 3-6). The development of the CLI and Functional Testing suite (Weeks 5-7) will overlap with the algorithm work to enable early testing. The Web Front-end Implementation (Weeks 7-10) will then commence, building upon the now-validated backend. The subsequent phase, Experimental Analysis and Parameter Tuning (Weeks 10-11), will utilize the nearly complete system to fine-tune transfer penalties. The final weeks (Weeks 11-14) are dedicated to Documentation Finalization and Presentation Preparation, ensuring all reports (D3, D4, D5) are polished and the Pra-KID demonstration is thoroughly prepared. Major milestones are synchronized with deliverable deadlines at Weeks 3, 5, 8, 12, and 14.

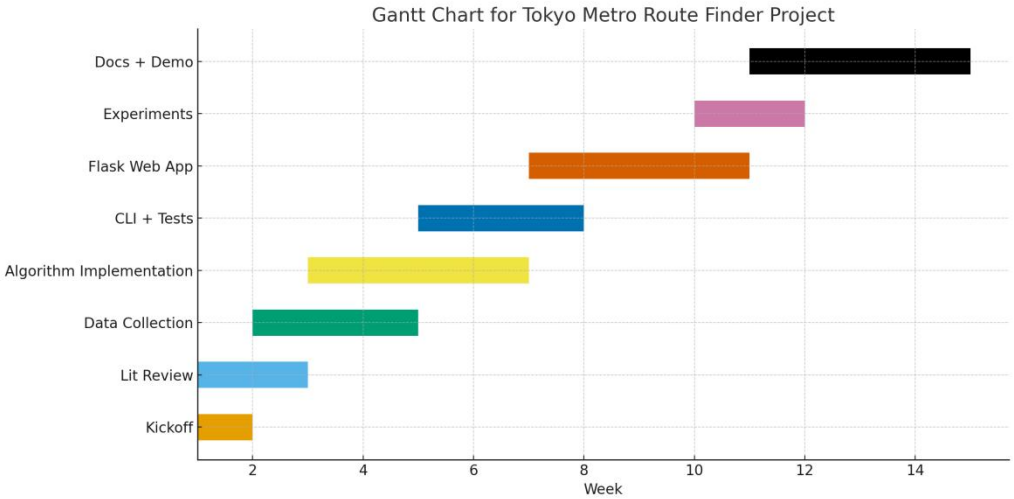


Figure 1.1 Gantt Chart

1.9 SUMMARY

In conclusion, this project plan outlines a viable and academically rigorous pathway to develop a prototype Tokyo Metro Route Finder. By leveraging the well-established Dijkstra's algorithm within a modern software engineering framework, the project addresses a tangible real-world problem while fulfilling the core requirements of the Computer Science curriculum. The incremental development approach, clear scope definition, and structured timeline provide a robust framework for success. The final deliverable will be a demonstrable, documented, and extensible system that not only serves as a proof-of-concept for time-based metro navigation but also provides a solid foundation for future academic and practical exploration in the domain of intelligent transportation system.

CHAPTER II

LITERATURE REVIEW

2.1 INTRODUCTION

Urban rail transit systems, especially those in megacities such as Tokyo, represent some of the most complex transportation infrastructures in the world. As ridership increases and network topology becomes more interconnected, the demand for efficient and transparent route-planning systems has also grown. While commercial applications—such as Google Maps, Tokyo Metro's official route planner, and NAVITIME—can compute optimal travel routes, they function as closed-source platforms that do not disclose the underlying algorithms or path-selection criteria. This creates a substantial limitation for educational, research, and prototyping purposes, where algorithmic transparency and parameter experimentation are crucial.

In academic contexts, Dijkstra's algorithm has long been regarded as a foundational method for single-source shortest-path computation in weighted graphs with non-negative edges. However, many academic demonstrations still rely on highly simplified, synthetic graphs that do not reflect the complexity of actual metro systems. Real-world considerations such as multi-line interchange stations, transfer walking times, and dynamic route costs remain underexplored in educational implementations. Recent literature also highlights the increasing use of GTFS (General Transit Feed Specification) datasets, advanced algorithms such as A* and RAPTOR, and passenger-behavior modeling, yet there remains limited work on combining these elements into a transparent, accessible prototype.

This literature review synthesizes key findings from recent studies (2015–2024), compares their methodologies, and identifies the research gaps this project aims to address. The focus is on metro route optimization, transfer modeling, open data

frameworks, and algorithmic transparency—core components of the proposed Tokyo Metro Route Finder using Dijkstra's Algorithm.

2.2 CURRENT RESEARCH AND RELATED TECHNOLOGIES

Past studies over the last decade have offered a wide range of insights into route-planning algorithms, metro network modeling, and transfer-time optimization. Collectively, these works highlight both the maturity of shortest-path research and the persistent challenges that arise when applying classical algorithms to real-world transportation systems.

One consistent finding is that Dijkstra's algorithm remains a reliable and foundational approach for computing shortest paths in transit networks. Despite the emergence of more advanced models, such as Connection Scan Algorithms, RAPTOR, and various A*-based methods, researchers continue to rely on Dijkstra as a benchmark due to its deterministic optimality, mathematical simplicity, and suitability for weighted graphs with non-negative travel times. Numerous studies between 2016 and 2022 confirm that Dijkstra, when implemented with a binary heap or Fibonacci heap, achieves sufficient performance for metro-scale networks comprising a few hundred stations.

Another major theme is the increasing importance of transfer modeling. Earlier studies often oversimplified metro routing by treating all edges as equal transitions, ignoring the fact that passengers incur additional time when switching between lines. More recent research demonstrates that transfer time—comprising walking distance between platforms, vertical movement through escalators, and waiting for the next train—can significantly influence total travel time. Several authors argue that failure to incorporate transfer penalties leads to unrealistic route recommendations, particularly in highly interconnected networks like Tokyo, Beijing, or Seoul. This shift marks a growing awareness that traditional shortest-path algorithms must be adapted to better represent real passenger experience.

Studies have also emphasized the rise of GTFS (General Transit Feed Specification) as the standard for transit data modeling. Researchers increasingly use GTFS to construct accurate representations of bus, tram, and metro networks, as it provides a well-structured format for stations, routes, schedules, and transfers. The adoption of GTFS facilitates comparative studies across cities by providing normalized datasets. Even when official GTFS data is unavailable—such as in some sections of the Tokyo Metro—researchers often create GTFS-like synthetic datasets to reproduce real-world conditions. This trend demonstrates the field's shift toward data-driven modeling rather than purely theoretical graph construction.

Additionally, literature from 2020 onwards reveals a growing interest in behavior- and context-aware routing, where the shortest path is no longer determined strictly by travel time but also by passenger behavior, congestion levels, and qualitative preferences such as minimizing transfers or avoiding crowded lines. Machine learning models have been used to estimate realistic transfer times or predict crowding levels during peak hours. Although these approaches fall outside the scope of simple prototypes, they highlight the long-term direction of intelligent transportation research and the increasing integration of human-centric considerations.

Finally, past studies show that while many routing prototypes aim for algorithmic sophistication, few prioritize transparency, educational clarity, and modularity. Many state-of-the-art works focus on performance at the expense of interpretability, making them unsuitable for teaching or beginner-level experimentation. At the same time, commercial route planners offer powerful functionality but conceal their algorithms, making them inaccessible for academic understanding. This gap indicates a need for systems that combine practical realism with educational value—systems that provide clear data structures, visible algorithmic operations, and customizable parameters such as transfer penalties.

Overall, the body of literature reveals a mature but evolving field where classical algorithms still play a central role, real-world data modeling is increasingly standardized, and user-centric factors such as transfers and behavior are gaining attention. These findings directly motivate the development of a transparent and pedagogically oriented Tokyo Metro route-finding prototype.

2.3 COMPARISON OF PAST STUDIES

The following table summarizes key dimensions where past studies differ, providing a clearer overview of methodological diversity in the field.

Dimension	Findings from Past Studies	Implication for Current Project
Algorithms Used	Dijkstra widely used; A*, RAPTOR, CSA emerging for large transit networks	Dijkstra appropriate for prototype; A* can be future extension
Data Source	Increasing reliance on GTFS; some studies use synthetic metro data	Project may use custom CSV + GTFS-inspired structure
Transfer Modeling	Advanced studies incorporate detailed penalties; older studies ignore	Project must include transfer penalties for realism
Network Scale	Studies range from small synthetic networks to full metro systems	Tokyo Metro's size manageable for Dijkstra
Real-Time Features	Advanced studies integrate GTFS-RT; uncommon in educational systems	Real-time features out of scope but potential future work

Transparency	Commercial systems lack transparency; research systems may focus on performance rather than clarity	Current project focuses on algorithmic transparency
Interface & Usability	Research prototypes often lack user-friendly interfaces	Project includes CLI + web UI for demonstration
Educational Value	Few projects balance realism with simplicity	This project bridges that gap

Table 2.1 Comparison of Past Studies (2015–2024)

This comparison demonstrates that while advanced routing systems exist, few combine realistic metro modeling, transfer penalties, and transparent algorithms within a student-accessible prototype.

2.4 RESEARCH GAPS

Despite considerable progress in transit routing research, three major gaps remain evident in recent studies.

First, most existing route-planning systems—both commercial and academic—lack algorithmic transparency. Commercial applications do not disclose how routes are computed, while academic works often focus on theoretical performance without providing accessible, modular prototypes. This leaves a gap for educational systems that allow students to observe, test, and modify the shortest-path logic in a realistic metro context.

Second, many past studies and student-level implementations oversimplify metro networks by ignoring or inadequately modeling transfer penalties. Although recent literature highlights that transfers significantly influence total travel time and passenger decision-making, few openly available prototypes integrate configurable transfer costs or reflect the complexity of real interchange stations, especially in networks as dense as Tokyo Metro.

Third, there is a lack of open, lightweight datasets and prototypes specifically tailored to the Tokyo Metro system. While GTFS has become the global standard for transit data, Tokyo Metro's publicly available data is limited, and existing datasets often require extensive processing. As a result, researchers and students lack a clean, ready-to-use dataset and accompanying system for experimenting with algorithms such as Dijkstra in a real-world Tokyo context.

These gaps collectively justify the development of a transparent, transfer-aware, and dataset-friendly Tokyo Metro Route Finder prototype.

2.5 CONCLUSION

In summary, the past decade of research has provided substantial advancements in algorithmic routing, transit data modeling, and metro network analysis. However, the literature consistently shows that existing systems—whether commercial or academic—do not fully meet the combined needs of transparency, realism, and educational usability. Classical algorithms such as Dijkstra remain highly relevant for metro-scale networks, yet many implementations rely on overly simplified synthetic data that fail to capture essential real-world elements, particularly transfer penalties and the structural complexity of large interchange stations. At the same time, advanced routing models and GTFS-based approaches, while powerful, are often too complex or inaccessible for student-level learning and rapid prototyping.

These findings highlight an opportunity to develop a system that bridges the gap between academic theory and practical application. By constructing a transparent, transfer-aware, and data-driven prototype tailored to the Tokyo Metro network, this project aims to contribute an accessible platform that supports both learning and experimentation. Such a system not only reflects the core insights from existing research but also provides a strong foundation for future extensions, including A heuristics, real-time GTFS integration, and interactive visualization. Through this approach, the project positions itself as both a response to current research limitations and a stepping stone toward more advanced work in intelligent transit-routing systems.

CHAPTER III

REQUIREMENTS ANALYSIS AND SPECIFICATION

3.1 INTRODUCTION

This chapter presents the detailed requirements analysis for the proposed system, Tokyo Metro Route Finder, which computes the shortest-time route between any two stations in the Tokyo Metro network using Dijkstra's algorithm. Requirements Analysis is a critical stage in the system development cycle, as it transforms the project's aims into clear, structured, and verifiable specifications.

The analysis in this chapter follows the Structured Analysis Approach, which is ideal for computational and data-driven systems. Structured Analysis emphasizes the decomposition of processes, the modeling of data flows, and the representation of system boundaries and interactions. This makes it a suitable approach for algorithm-centric projects such as route-finding, where the transformation of inputs into outputs through defined processes is the system's main focus.

The chapter is divided into several sections: system context, functional and non-functional requirements, data flow models (DFD Level 0 and Level 1), process specifications for Dijkstra's algorithm, a detailed data dictionary, user characteristics, assumptions, constraints, and the overall summary. Collectively, these elements establish a comprehensive and traceable foundation for implementing the system in later stages.

3.2 DEFINITION OF USER NEEDS

The System Context Diagram (SCD) provides a high-level view of the system's scope and interactions with external entities. It defines the system boundary and illustrates the flow of data between the Tokyo Metro Route Finder System (TMRFS) and the users.

End User (Web Interface User)	**Simple Station Selection：**Users must be able to select origin and destination stations easily, preferably from a list or dropdown that minimizes typing and prevents input errors. **Fast and Accurate Route Computation：** General users expect immediate results. They require the system to compute the shortest-time route within a few seconds and present the results clearly. **Clear and Understandable Output：** The route should be presented in a structured, step-by-step format that includes: Sequence of stations Line names Transfer points Total travel time This helps users plan their journey without confusion. **Accessible and Device-Friendly Interface：** The system must work on standard web browsers without the need to install software. Pages should render clearly on both desktop and mobile screens.
Technical User (CLI User)	Uses the command-line interface to run tests or verify paths. Receives detailed algorithm output for debugging and development.

Table 3.1: Definition Of User Needs

The Tokyo Metro Route Finder System (TMRFS) receives station input from users, loads graph data from CSV files, runs Dijkstra's algorithm, and returns the optimal path.

3.3 System Requirement Specifications

This section defines the complete System Requirement Specifications (SRS) for the proposed *Tokyo Metro Route Finder System*. The specifications include detailed functional requirements mapped to user needs, non-functional (quality) and domain-specific requirements, as well as hardware and software requirements for both system development and deployment. Since the project is not primarily a network or cybersecurity system, network-related requirements are addressed at an application communication level.

3.3.1. Mapping User Needs to System Functional Requirements

Based on the user needs identified in Section 3.2, the following table maps user functional requirements to system-level functional requirements, ensuring traceability and completeness.

User Need	**System Functional Requirement**
Select origin and destination stations	The system shall provide a validated station selection mechanism via Web UI and CLI
Obtain shortest-time route	The system shall compute the shortest-time path using Dijkstra's algorithm
View transfer information	The system shall detect and display line changes and transfer stations
Fast response time	The system shall return results within one second under normal conditions
Algorithm transparency (technical users)	The system shall expose detailed execution results via CLI
Error-free usage	The system shall validate inputs and display meaningful error messages

Table 3.2: User Needs vs System Functional Requirements

3.3.2. System Functional Requirements Specification

This subsection specifies the system-level functional requirements in a formal and structured manner.

ID requirement	Requirement
FRS-1 Input Handling	The system shall accept origin and destination station inputs from both Web Interface users and Command-Line Interface users.
FRS-2 Input Validation	The system shall verify that the selected stations exist in the dataset before computation begins.
FRS-3 Data Loading	The system shall load metro station and connection data from structured CSV files.
FRS-4 Graph Construction	The system shall transform metro data into a weighted graph representation suitable for shortest-path computation.
FRS-5 Pathfinding Algorithm Execution	The system shall execute Dijkstra's algorithm to compute the shortest travel-time path between two stations.
FRS-6 Transfer Penalty Processing	The system shall apply a configurable transfer time penalty when the route requires changing metro lines.
FRS-7 Path Reconstruction	The system shall reconstruct the full path from the algorithm's predecessor table.
FRS-8 Result Presentation	The system shall present the computed route, total travel time, and transfer details in a readable format.
FRS-9 Dual Interface Support	The system shall support both Web-based and Command-Line-based interactions.
FRS-10 Error Handling	The system shall detect invalid inputs or unreachable routes and display appropriate error messages.

Table 3.3: System Functional Requirements

3.3.3. Domain Requirements

Domain requirements arise from the urban rail transportation domain and constrain system behavior.

- Travel times between stations are assumed to be non-negative values.
- Transfer penalties must reflect realistic passenger transfer behavior.
- Routes must follow actual metro line connectivity.
- The system must treat interchange stations as valid transfer points.
- The system output is advisory and not a real-time navigation service.

3.3.5. Hardware And Software Requirements For System Developers

1. Hardware

Component	**Requirement**
Processor	Intel i5 / AMD Ryzen 5 or equivalent
Memory	Minimum 8 GB RAM
Storage	At least 10 GB free disk space
Display	1366 × 768 resolution or higher
Network	Internet connection (for Web Interface access)

Table 3.4: Hardware Requirements For System Developers

2. Software

Software	**Purpose**
Python 3.x	Core algorithm and backend development
Flask	Web application framework
VS Code / PyCharm	Development environment
Git	Version control
CSV Libraries	Data processing

Table 3.5: Software Requirements For System Developers

3.3.5. Hardware And Software Requirements For System Users

1. Hardware

Component	Requirement
Processor	Any modern CPU
Memory	4 GB RAM
Browser	Chrome, Edge, Firefox, or Safari
Network	Internet connection (for Web Interface access)

Table 3.6: Hardware Requirements For System Users

2. Software

Software	Purpose
Python Runtime	System execution
Web Browser	User interaction
Flask Server	Application hosting

Table 3.7: Software Requirements For System Users

This section defined the complete System Requirement Specifications for the Tokyo Metro Route Finder System. By clearly specifying functional, non-functional, domain, hardware, and software requirements, the system's development and deployment constraints are well established. These specifications ensure that the system is feasible, reliable, and aligned with both academic objectives and real-world application constraints.

3.4 System Model

This section presents the System Model of the proposed Tokyo Metro Route Finder System. The system model is developed using the Structured Analysis Approach, which focuses on data flow, process decomposition, and execution sequence. This approach is selected because the system is algorithm-centric and primarily involves transforming user input data into optimized route outputs through a series of well-defined computational processes.

The system model is represented using three complementary models:

System Context Diagram

Data Flow Diagrams (DFD Level 0 and Level 1)

System Flowchart

Together, these models provide a clear and comprehensive representation of the system's scope, internal processing logic, and execution order.

3.4.1 System Context Diagram

The System Context Diagram defines the overall boundary of the Tokyo Metro Route Finder System and identifies its interaction with external entities. At this level, the internal workings of the system are abstracted, and only high-level data exchanges are shown.

External Entities

End User (Web Interface User)

Provides origin and destination station inputs and receives route computation results.

Technical User (CLI User)

Submits station identifiers and receives detailed route output for testing and debugging.

Metro Data Repository

Supplies static datasets (stations.csv and edges.csv) containing station and connection information.

System Interaction

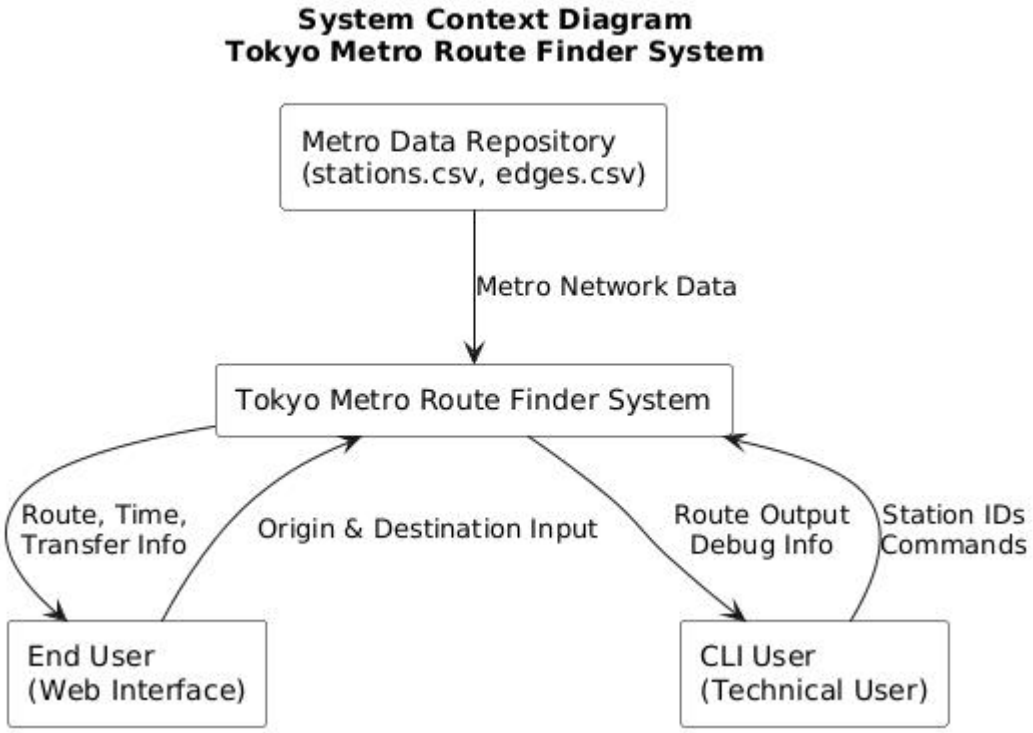
Users send station selection data to the system.

The system retrieves metro network data from CSV files.

The system outputs the shortest route, total travel time, and transfer details.

Figure 3.1 illustrates the System Context Diagram of

the Tokyo Metro Route Finder System



The diagram defines the overall system boundary and shows how the system interacts with external entities. The end user provides origin and destination station inputs to the system and receives the computed optimal route information. The system also retrieves metro network data from an external data repository containing station and connection information. This diagram clearly establishes the scope of the system and its external data interactions.

3.4.2 Data Flow Diagram (DFD Level 0)

The DFD Level 0 provides a high-level logical view of how data flows through the system. At this level, the entire system is represented as a single process.

Main Process

Process 0: Compute Shortest Route

Data Flow Description

Input: Origin and destination station information from users.

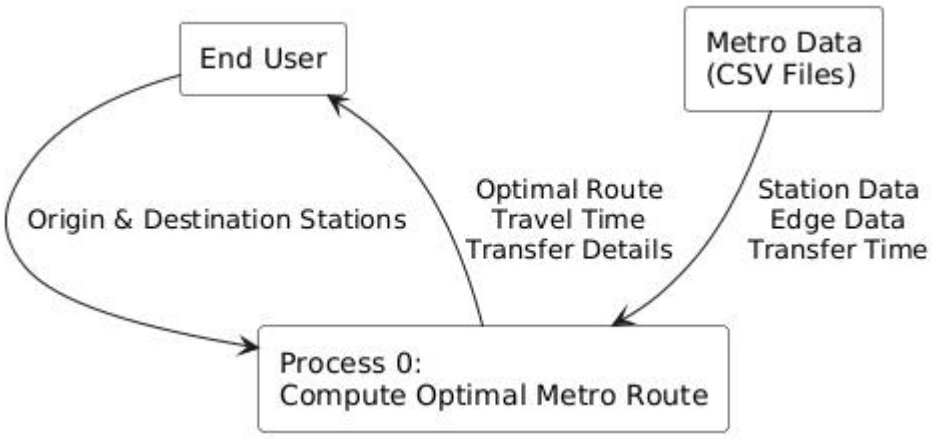
Processing: Shortest route computation based on travel time.

Output: Route sequence, total travel time, and transfer information.

Data Store: Metro network datasets (stations.csv, edges.csv).

This diagram emphasizes the transformation of user input into meaningful output while highlighting the system's dependency on structured data.

Figure 3.2 shows the DFD Level 0 of the system.



This diagram represents the Tokyo Metro Route Finder System as a single high-level process. User input and metro network data are transformed by the system into optimal routing results. The diagram emphasizes the primary data flows without exposing internal processing details, which is consistent with the objectives of Level 0 data flow modeling.

3.4.3 Data Flow Diagram (DFD Level 1)

The DFD Level 1 decomposes the main process in DFD Level 0 into a set of interconnected subprocesses, providing greater detail on how the system operates internally.

Process Decomposition

P1: Read and Validate Input

Receives station selections from users and verifies their existence in the dataset.

P2: Load Metro Network Data

Reads station and connection data from CSV files and constructs a weighted graph representation of the metro network.

P3: Execute Shortest Path Algorithm

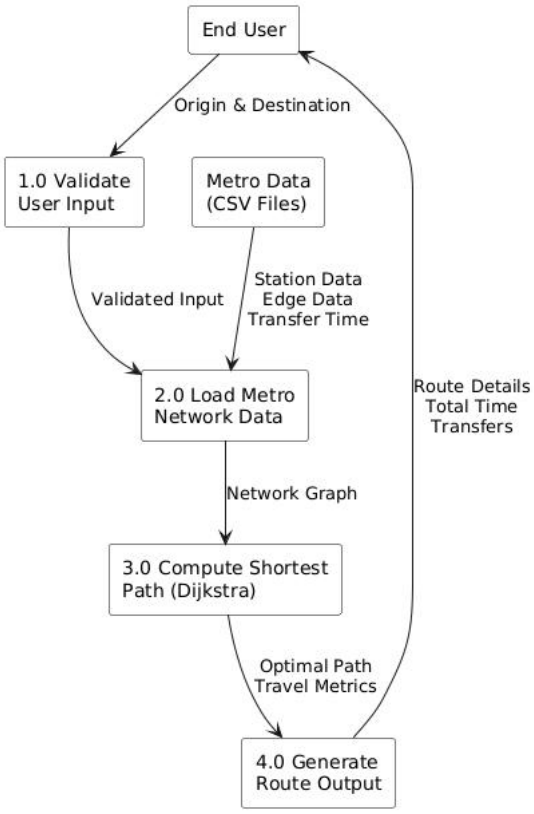
Applies Dijkstra's algorithm to compute the shortest travel-time path while incorporating transfer penalties for line changes.

P4: Generate and Display Output

Reconstructs the optimal route and formats the results for presentation via the Web Interface or Command-Line Interface.

The Level 1 DFD clearly demonstrates the logical sequence of data processing and ensures traceability between system requirements and implementation logic.

Figure 3.3 illustrates the DFD Level 1 of
the Tokyo Metro Route Finder System



The diagram decomposes the main system process into four logical subprocesses: input validation, data loading, shortest-path computation, and result generation. This decomposition provides a clearer view of how data is progressively transformed within the system while maintaining a structured and hierarchical analysis approach.

3.4.4 System Flowchart

The System Flowchart represents the execution order of processes within the system. Unlike DFDs, which emphasize data movement, the flowchart focuses on the control flow and decision-making logic.

Flow Description

The system starts and waits for user input.

The user enters origin and destination stations.

The system validates the input.

If the input is invalid, an error message is displayed and the process terminates.

If valid, the system loads the metro dataset.

The system constructs the weighted graph.

Dijkstra's algorithm is executed.

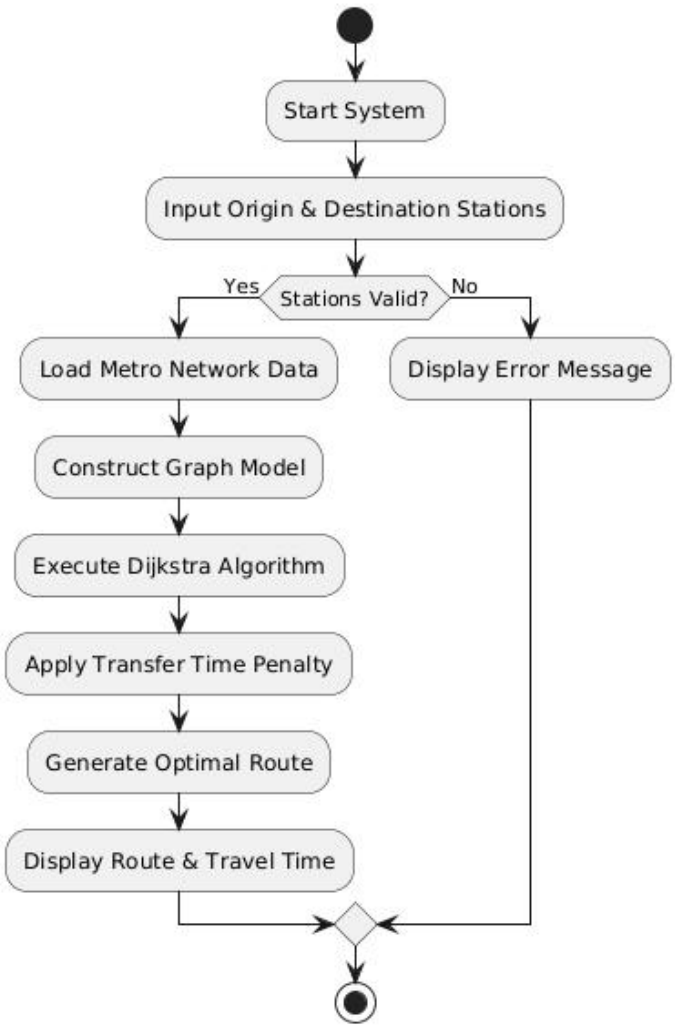
Transfer penalties and total travel time are calculated.

The shortest route is reconstructed.

The final route information is displayed to the user.

This flowchart ensures that the system's operational logic is clearly defined and aligns with the functional requirements.

Figure 3.4 shows the System Flowchart of
the Tokyo Metro Route Finder System.



The flowchart illustrates the sequential execution of system processes from user input to result presentation. Decision points are used to handle invalid input scenarios, ensuring system reliability. The flowchart clearly represents the logical flow of operations, including data loading, graph construction, and shortest-path computation.

3.5 SUMMARY

This chapter has presented a comprehensive System Requirements Specification and System Model for the Tokyo Metro Route Finder System using a structured analysis approach. The analysis began with the identification of user needs and system requirements, clearly defining both functional and non-functional aspects of the system to ensure that the proposed solution addresses practical user expectations and operational constraints.

The system modeling process was carried out using standard structured analysis techniques, including a System Context Diagram, Data Flow Diagrams (Level 0 and Level 1), and a System Flowchart. The System Context Diagram established the system boundaries and external interactions, while the DFD Level 0 provided a high-level view of the main data transformation process. The DFD Level 1 further decomposed the system into detailed subprocesses, illustrating how user input and metro network data are processed to generate optimal routing results. The System Flowchart complemented these diagrams by depicting the sequential execution and decision logic of the system.

Together, these models provide a clear and logical representation of the system's structure, data flow, and operational behavior. They serve as a solid foundation for subsequent design and implementation phases by ensuring traceability between user requirements and system processes. This structured and methodical approach enhances system clarity, reduces development risk, and supports the successful realization of the Tokyo Metro Route Finder System in the later stages of the project.

CHAPTER IV

DESIGN SPECIFICATION

4.1 INTRODUCTION

This chapter presents the Design Specification for the Tokyo Metro Route Finder System. Building upon the System Requirements Specification and structured system models defined in Chapter 3, this chapter focuses on translating system requirements into a concrete and implementable design. The design specification outlines the system architecture, data design, algorithm design, and interface design, ensuring that all functional and non-functional requirements are addressed in a systematic and traceable manner.

The design adopts a structured and modular approach to enhance system maintainability, scalability, and clarity. Each design component is aligned with the processes identified in the Data Flow Diagrams and System Flowchart, ensuring consistency between system analysis and system design.

4.2 ARCHITECTUAL DESIGN

4.2.1 Overall Architecture

The Tokyo Metro Route Finder System is designed using a layered architectural model. This architecture separates concerns into distinct layers, enabling easier development, testing, and future extension.

The system consists of four main layers:

Presentation Layer: Handles user interaction through a Command-Line Interface (CLI) and a web-based interface.

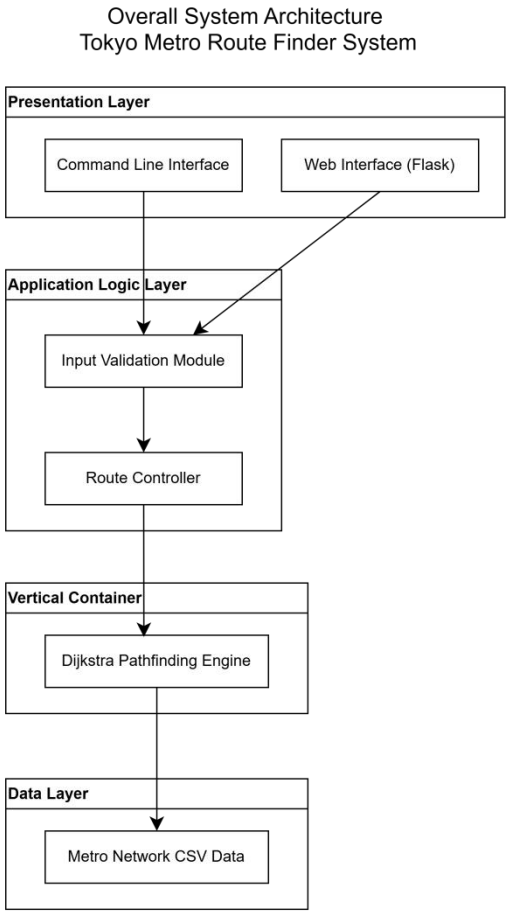
Application Logic Layer: Coordinates data processing, input validation, and routing logic.

Algorithm Layer: Implements the shortest-path computation using Dijkstra's algorithm with transfer time penalties.

Data Layer: Manages the storage and retrieval of metro network data from structured CSV files.

This layered design ensures that changes in one layer (e.g., interface redesign) do not significantly affect other layers.

Figure 4.1 illustrates the overall system architecture of the Tokyo Metro Route Finder System



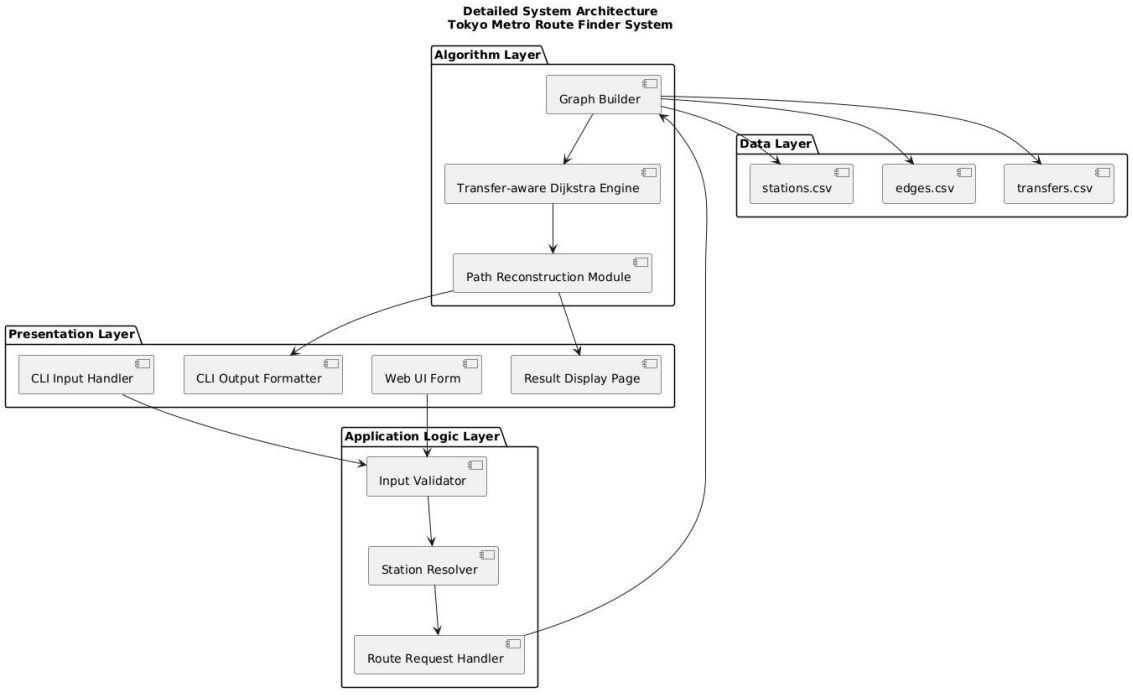
The architecture is organized into four logical layers: Presentation Layer, Application Logic Layer, Algorithm Layer, and Data Layer. This layered structure improves modularity and ensures clear separation of concerns between user interaction, control logic, algorithm execution, and data management.

4.2.2 Architectural Component Description

Table 4.1 Architectural Component Description

Component	Description
User Interface	Accepts origin and destination input and displays routing results
Input Validator	Ensures station names and IDs are valid
Route Engine	Coordinates graph construction and path computation
Dijkstra Module	Computes shortest path with transfer penalties
Data Manager	Loads and parses metro data from CSV files

Figure 4.2 presents the detailed system architecture of the Tokyo Metro Route Finder System



The diagram expands the layered architecture by illustrating internal components and their interactions. User input is validated and processed by the application logic layer before being passed to the algorithm layer, where the metro network graph is constructed and the transfer-aware Dijkstra algorithm is executed.

The computed route is then reconstructed and presented to users through both command-line and web interfaces. This detailed design supports maintainability, scalability, and traceability to system requirements.

4.3 DATABASE DESIGN

4.3.1 Class Diagram

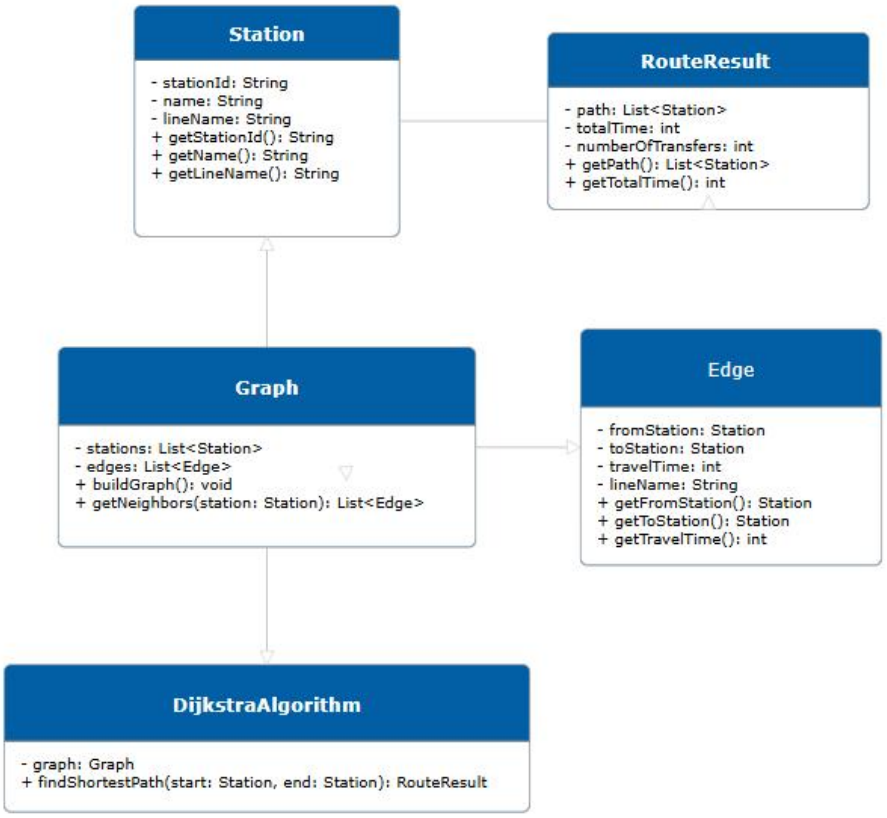
The data design of the Tokyo Metro Route Finder System defines how metro network information is structured, stored, and accessed by the system. A well-organized data design is essential to support efficient route computation, ensure data consistency, and maintain clarity between system components. In this project, a file-based data storage approach using Comma-Separated Values (CSV) files is adopted due to its simplicity, portability, and suitability for prototype-level development.

The system data design aligns with the structured analysis approach presented in Chapter 3, where data flows through validation, processing, and transformation stages. All data elements are designed to directly support the shortest-path computation process and result presentation.

Figure 4.3 illustrates a high-level design diagram of the Tokyo Metro Route Finder System, emphasizing the interaction between the user, user interface, and internal system components.

The user interacts with the system exclusively through the User Interface, which forwards routing requests to the core system. Internal components such as the Data Manager, Metro Graph, and Dijkstra Engine collaboratively process the request and generate routing results. This design improves system clarity by separating user interaction from internal data processing and algorithm execution.

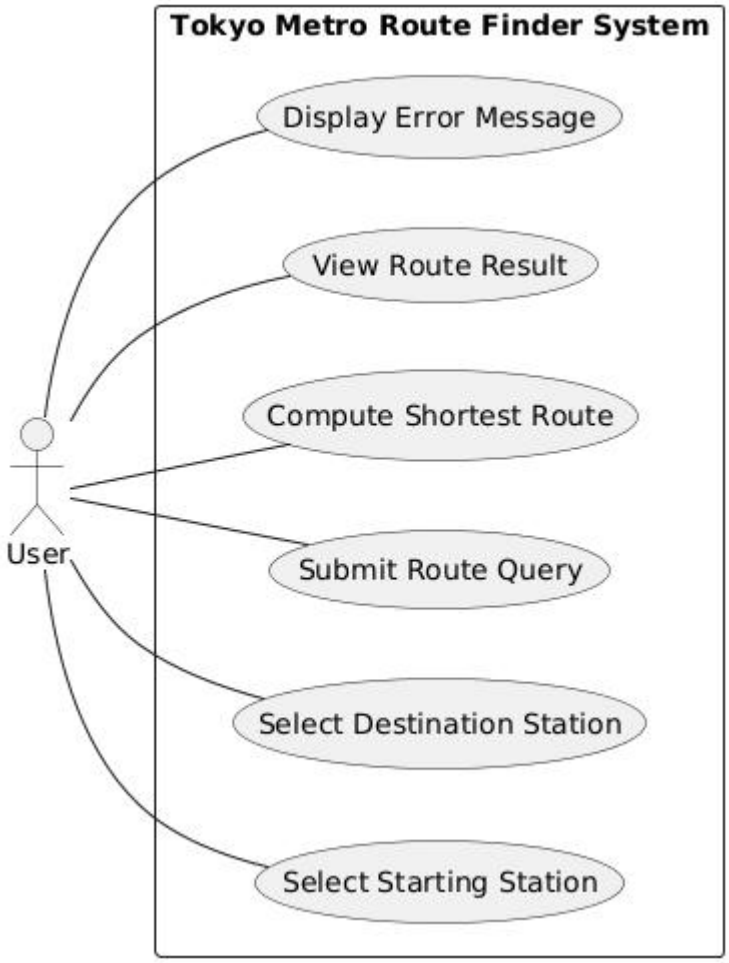
Figure 4.3.1 Large Class Diagram



This diagram is included at the design stage to improve understanding of data flow and component interaction, rather than to model object-oriented behavior.

Use Case Diagram

Figure 4.3.2: Use Case Diagram of the Proposed System



This section describes the class diagram of the proposed system, which models the object-oriented structure of the metro route finding application.

The Station and Edge classes represent the fundamental components of the metro network, including stations and the connections between them.

The Graph class maintains the overall network structure and provides access to neighboring stations.

The DijkstraAlgorithm class is responsible for computing the shortest route between two stations based on travel time.

The RouteResult class stores the computed path and related route information, which is returned to the user interface for display.

4.3.2 Data Storage Design

The system uses three primary CSV files to represent the Tokyo Metro network:

Table 4.2 shows Data Storage Design

Data File	**Description**
stations.csv	Stores information about metro stations
edges.csv	Defines connections between stations and travel times
transfers.csv	Stores transfer penalty information for interchange stations

These files collectively represent the metro network as a weighted graph, where stations are nodes and connections are edges.

4.3.3 System Data Dictionary

The system data dictionary defines each data element used by the Tokyo Metro Route Finder System. It specifies data names, descriptions, data types, and constraints to ensure consistency throughout the system.

Table 4.3 Data Dictionary for stations.csv

Field Name	**Data Type**	**Description**	**Constraints**
station_id	Integer	Unique identifier for each station	Primary key, not null
station_name	String	Official station name	Not null
line_name	String	Metro line associated with the station	Not null
latitude	Float	Geographical latitude of the station	Optional
longitude	Float	Geographical longitude of the station	Optional

Table 4.4 Data Dictionary for edges.csv

Field Name	Data Type	Description	Constraints
from_station_id	Integer	Starting station identifier	Foreign key
to_station_id	Integer	Destination station identifier	Foreign key
travel_time	Integer	Travel time between stations (minutes)	> 0
line_name	String	Metro line for this connection	Not null

Table 4.5 Data Dictionary for transfers.csv

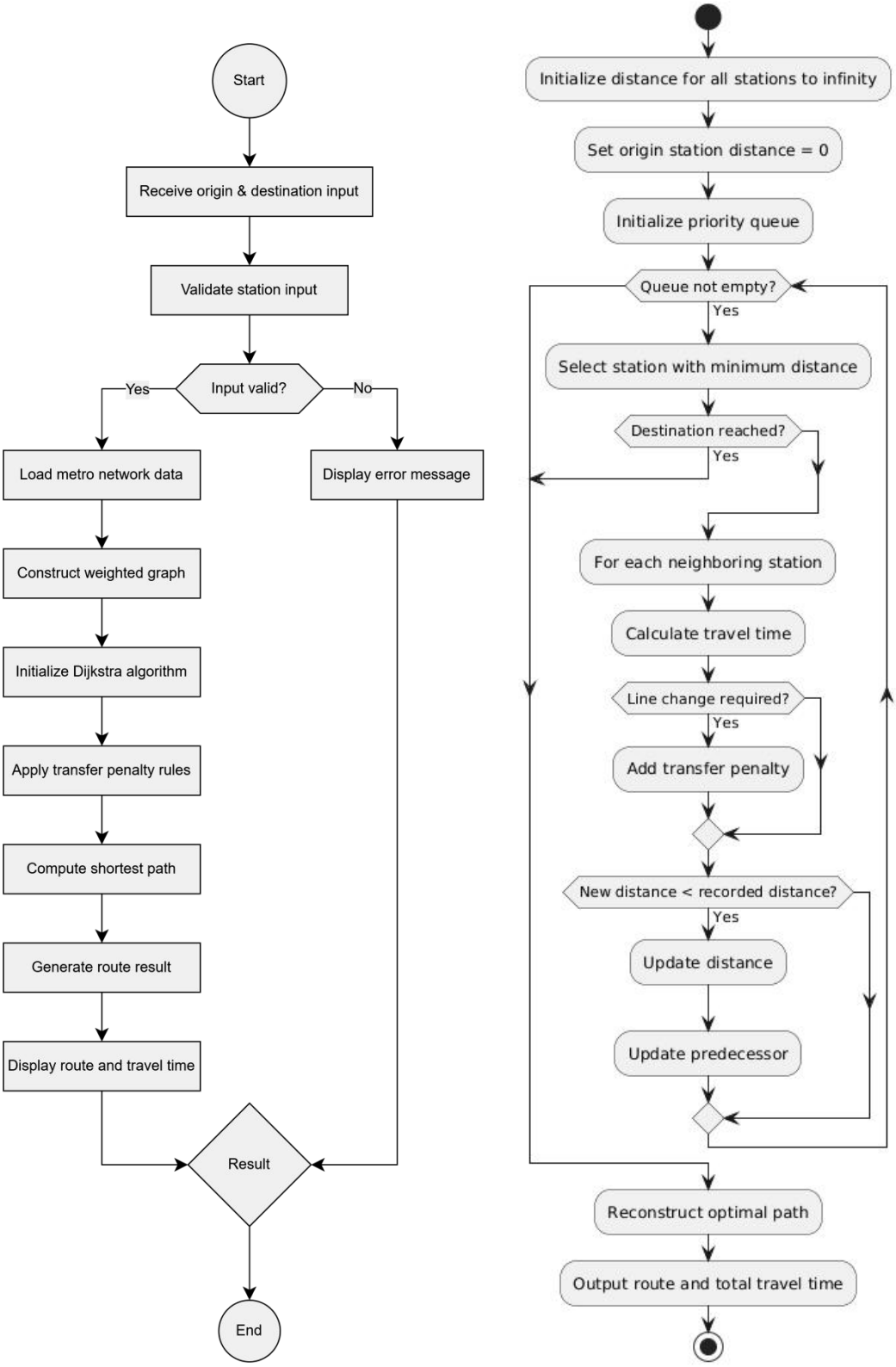
Field Name	Data Type	Description	Constraints
from_station_id	Integer	Starting station identifier	Foreign key
to_station_id	Integer	Destination station identifier	Foreign key
travel_time	Integer	Travel time between stations (minutes)	> 0
line_name	String	Metro line for this connection	Not null

The data design directly supports the transfer-aware Dijkstra algorithm used in the system. Edge weights represent travel time, while additional penalties from transfers.csv are applied when a line change occurs. This separation of travel time and transfer cost improves algorithm flexibility and allows easy parameter tuning during experimentation.

4.4 ALGORITHM DESIGN

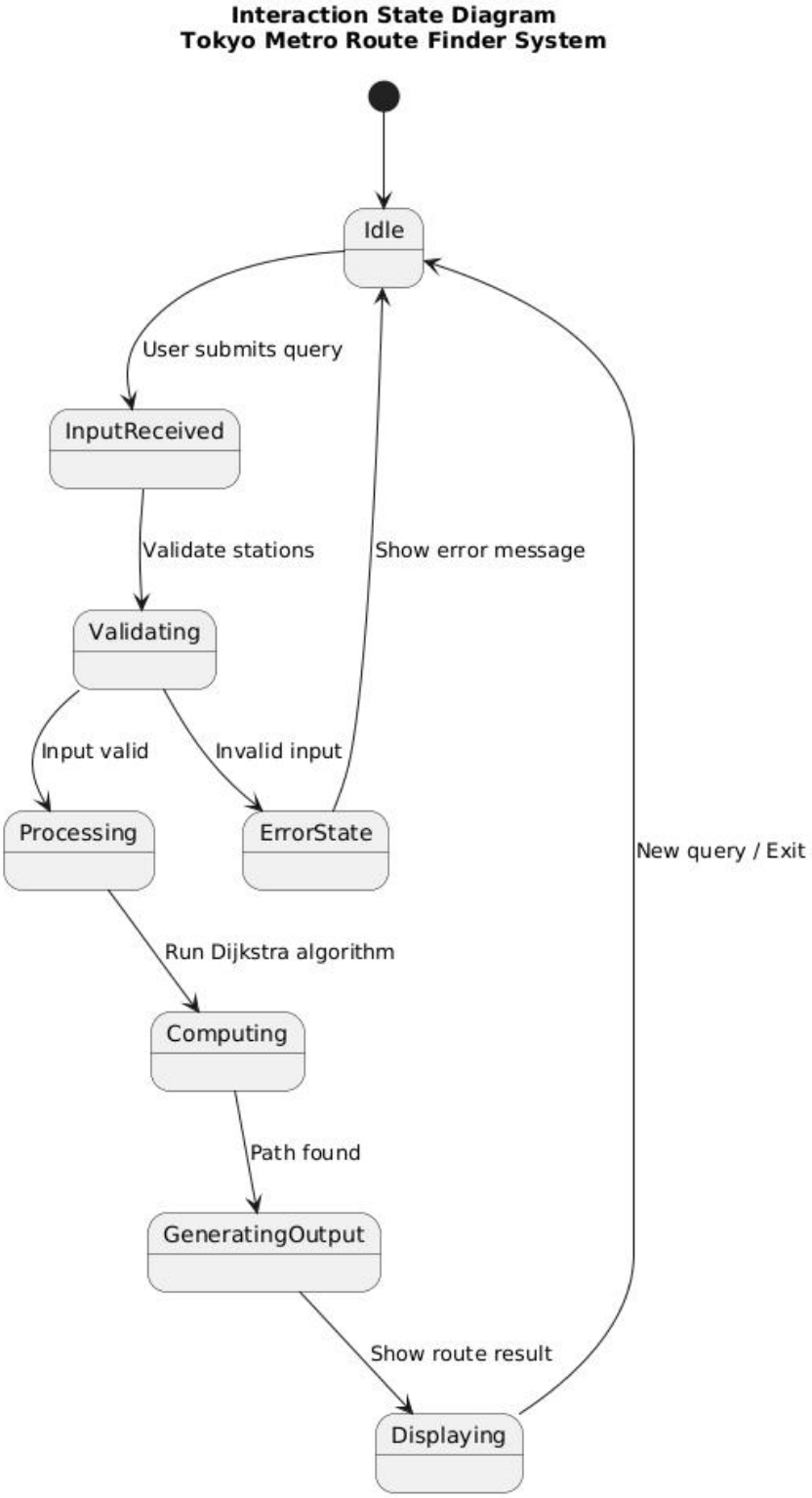
4.4.1 Algorithm Flowchart

Figure 4.4 Algorithm Flowchart



4.4.2 Statechart Diagram of The Interaction

Figure 4.5 illustrates the interaction state diagram of the Tokyo Metro Route Finder System during a route query process



The diagram represents the system's behavior as a series of states triggered by user interaction and internal processing events. The system transitions from an idle state to input validation, followed by route computation and result generation. Error states are included to handle invalid user input gracefully. This state-based representation clarifies how the system responds dynamically to user actions and algorithm execution outcomes.

Table 4.6 Explanation of Interaction States

Idle	The system remains in an idle state while waiting for the user to initiate a route query.
InputReceived	This state represents the moment when the system receives the origin and destination stations submitted by the user.
Validating	The system checks whether the input stations exist in the metro network dataset and verifies input correctness.
ErrorState	The system enters this state when invalid input is detected and prepares an appropriate error message for the user.
Processing	In this state, the system prepares the necessary data structures and initializes the route computation process.
Computing	The transfer-aware Dijkstra algorithm is executed to calculate the shortest travel-time path between the selected stations.
GeneratingOutput	The system constructs the final route details, including travel time and transfer information.
Displaying	The computed route and journey information are presented to the user through the selected interface.
Return to Idle	After displaying the results, the system returns to the idle state, ready to accept a new query.

4.5 INTERFACE DESIGN

The interface design of the Tokyo Metro Route Finder System aims to provide clear, simple, and efficient interaction for different types of users. To accommodate both technical and non-technical users, the system offers two interface modes: a Command-Line Interface (CLI) and a web-based graphical interface. The design prioritizes usability, clarity of information presentation, and consistency with the system's functional requirements.

The interfaces are designed to support the complete routing workflow, including input submission, route computation, and result visualization.

Figure 4.6 Route Query Entry Interface

Figure 4.5 shows the route query entry interface of the Tokyo Metro Route Finder System.

Figure 4.7 Route Information Navigation Interface

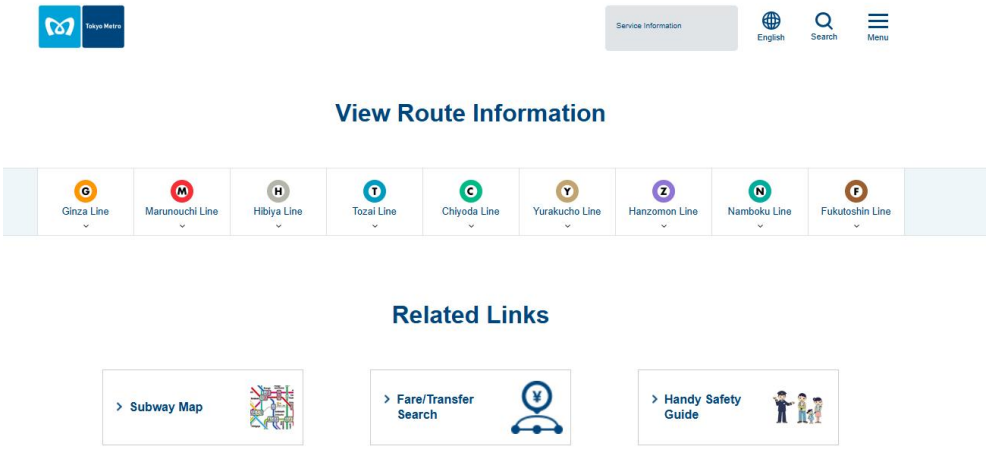


Figure 4.5.2 shows the route information navigation interface, which allows users to explore available metro lines and access route-related services before performing a detailed route query.

Figure 4.8 Route Query Input Interface

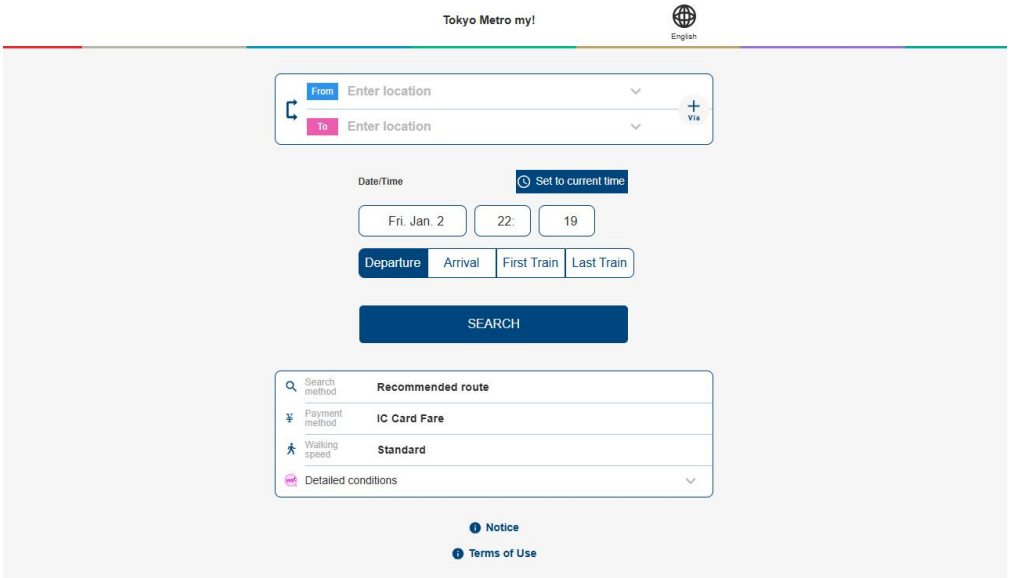


Figure 4.5.3 presents the route query input interface, where users enter the departure station, destination station, and travel time preferences to initiate a route search request.

Figure 4.9 Route Query Confirmation Interface

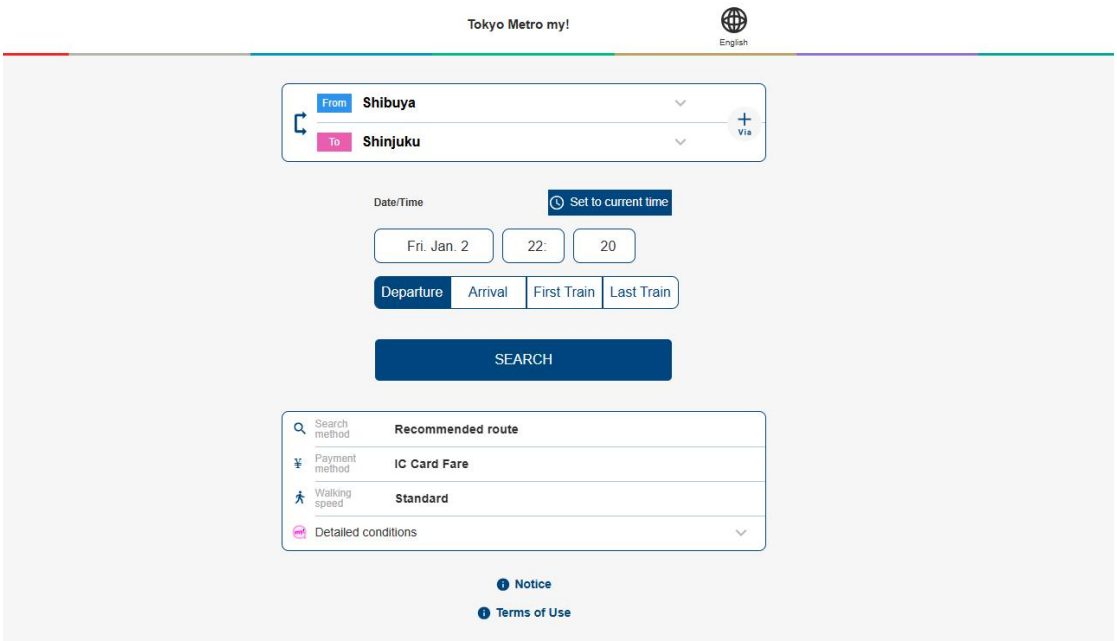


Figure 4.8 illustrates the route query confirmation interface, displaying the selected departure and destination stations along with the specified travel time before the route computation is executed.

Figure 4.10 Route Search Result Interface

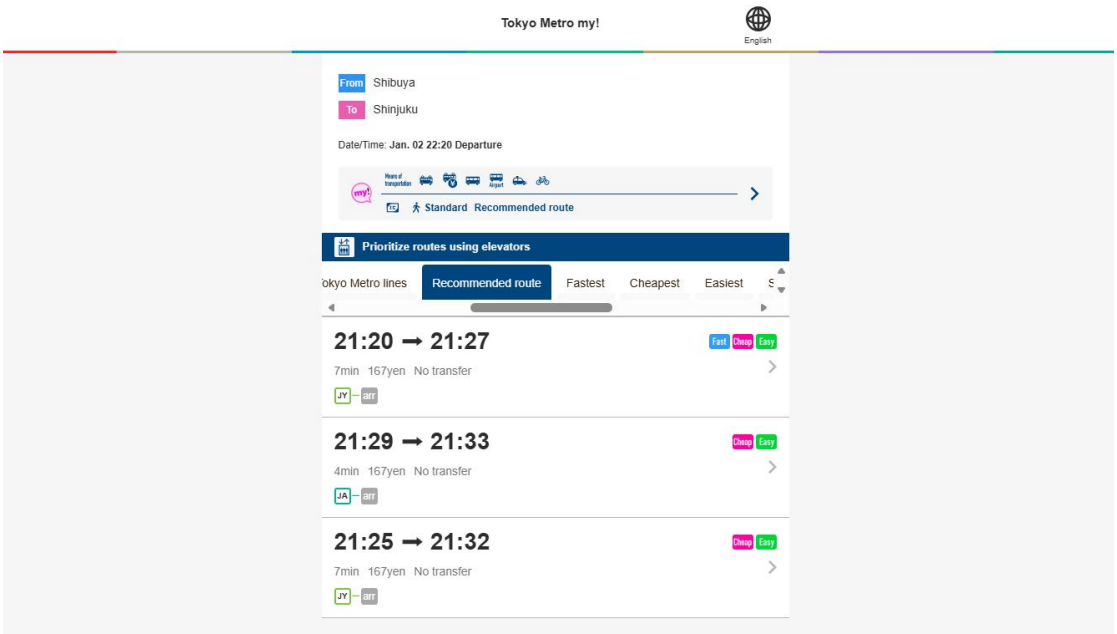


Figure 4.9 shows the route search result interface after the system completes the shortest-path computation.

Figure 4.11 Route Detail Interface

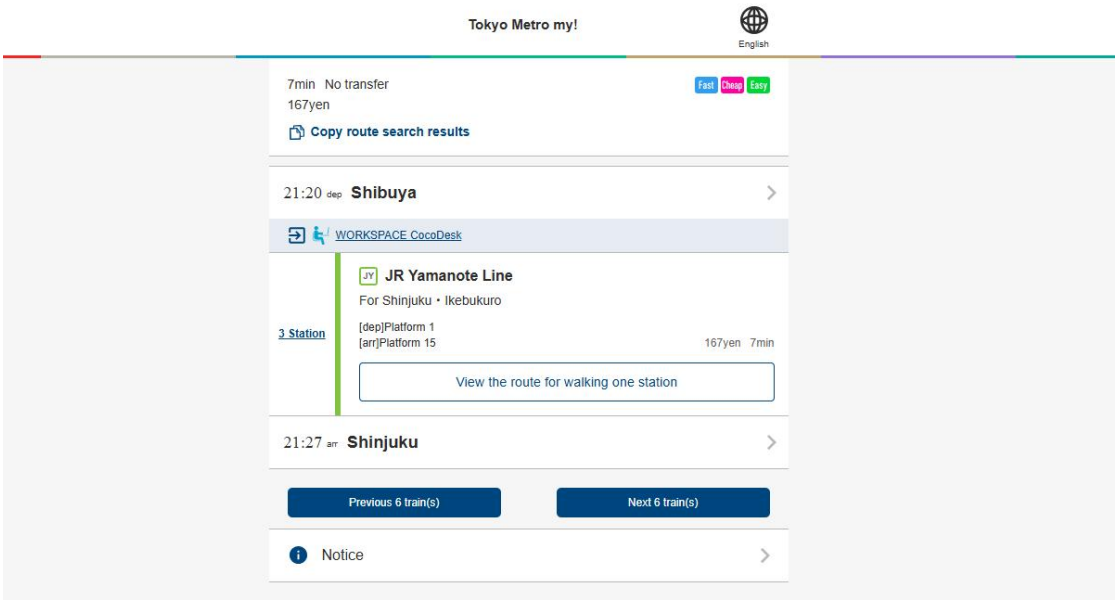


Figure 4.10 illustrates the route detail interface of the Tokyo Metro Route Finder System.

4.6 SUMMARY - Design Specification (D4)

In conclusion, Chapter 4 has presented the detailed system design of the Tokyo Metro Route Finder System, covering system architecture, data structure, algorithm design, and interface design. The structured design ensures that the system requirements defined in earlier chapters are effectively translated into a practical and implementable solution. By integrating a clear architectural model, well-defined data dictionary, efficient route-finding algorithms, and user-friendly interface layouts, the system is able to support accurate route queries and provide meaningful travel guidance to users. Overall, the design in this chapter establishes a solid technical foundation for system implementation and evaluation in the subsequent phase.

CHAPTER V

SYSTEM DEVELOPMENT

5.1 INTRODUCTION

This chapter describes the implementation phase of the Tokyo Metro Route Finder system, transitioning from the architectural designs outlined in the previous chapter to a fully functional software prototype. The implementation phase focuses on the actual construction of the system, where the proposed requirements and design specifications are translated into executable code and a user-oriented interface.

The primary purpose of this chapter is to provide a comprehensive record of the development process. It details the technical realization of the core pathfinding logic and the integration of the subway network data. By documenting the implementation details, this section ensures that the system's internal workings are transparent and aligned with the project’s initial objectives.

The scope of this implementation encompasses the selection of the development environment, the coding of the Dijkstra algorithm with its unique transfer penalty mechanism, and the creation of the web-based user interface using the Flask framework. It specifically highlights the critical components that allow the system to process complex transit data and provide optimized route recommendations to users.

5.2 DEVELOPMENT PROCESS

The implementation phase focuses on transforming the approved system design and requirements into a fully functional software system. The development activities were conducted in a structured, systematic, and iterative manner to ensure that the system meets functional, technical, and usability requirements.

The development process emphasizes modularity, maintainability, and scalability, allowing future improvements and extensions to be implemented efficiently. Throughout this phase, continuous testing and refinement were performed to identify errors early and improve overall system stability.

5.2.1 Development Process and Technology Used

The development of the proposed system was carried out using a structured and iterative development process, ensuring that the system was implemented in accordance with the approved requirements and design specifications. The primary goal of this development approach was to gradually transform the conceptual design into a functional and reliable system while maintaining flexibility for continuous improvement.

An iterative development methodology was adopted throughout the implementation phase. Instead of developing the entire system in a single cycle, the system was built incrementally through multiple development iterations. Each iteration focused on a specific set of functionalities, followed by testing and refinement. This approach allows early identification of design flaws, functional errors, and usability issues, thereby reducing development risk and improving system quality.

At the initial stage, the development environment was carefully prepared to support efficient implementation. This included selecting appropriate development tools, configuring the programming environment, and organizing the project structure in a modular manner. Proper environment setup ensured consistency across development and testing activities and minimized compatibility issues during later stages.

The system was implemented using a web-based architecture, following a client–server model. In this architecture, the backend is responsible for processing user requests, executing system logic, and managing data operations, while the frontend focuses on user interaction and presentation of information. This separation of responsibilities enhances system maintainability and allows independent modification of frontend and backend components.

For backend development, a lightweight and flexible framework was chosen to support rapid development and easy integration with data processing modules. The backend implementation emphasizes clarity of system logic, efficient request handling, and reliable data processing. The use of a high-level programming language enables faster development cycles and improves code readability.

Frontend development focuses on providing a user-friendly and intuitive interface. Standard web technologies were used to design responsive and accessible user interfaces. Special attention was given to layout consistency, clarity of information presentation, and ease of navigation, ensuring that users can interact with the system effectively without requiring technical expertise.

Data management is a critical component of the system development process. The system utilizes structured data storage mechanisms to ensure data consistency and integrity. Data processing workflows were designed to handle data loading, validation, transformation, and retrieval efficiently. This design supports both current system requirements and potential future expansion.

Throughout the development process, continuous testing and debugging were performed to verify system functionality and stability. Functional testing was conducted after each development iteration to ensure that newly implemented features operate as expected without affecting existing functionalities. This iterative testing strategy significantly improves system reliability.

Version control tools were used to manage code changes and track development progress. This practice enables systematic version tracking, facilitates rollback in case of errors, and supports collaborative development when required.

In summary, the development process combines an iterative methodology with a web-based system architecture and appropriate technology selection. This approach ensures that the system is not only functional and reliable but also scalable, maintainable, and suitable for future enhancements.

5.2.2 Critical Code Segments

Although the complete source code is not included in this document, selected critical code segments were identified and analyzed. These segments represent the core logic of the system and play a vital role in ensuring correct system functionality.

Critical code segments include:

System control flow and routing logic

Data processing and transformation functions

Core algorithm implementation

Exception and error handling mechanisms

These code segments were implemented with a focus on clarity, efficiency, and reliability. Detailed source code is provided in the final documentation (D7).

app.py

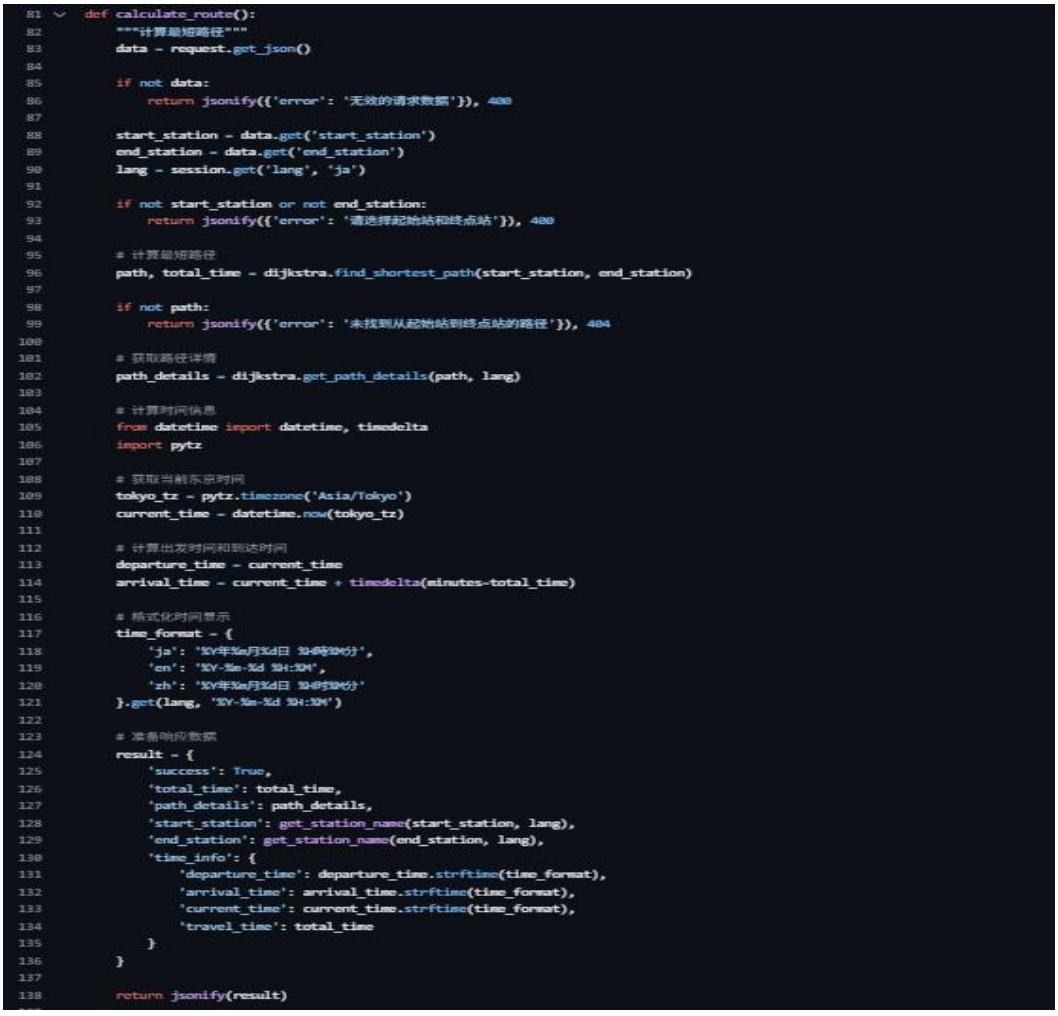
Figure 5.1 The Route Calculation Engine is the most critical endpoint. It bridges the gap between the raw Dijkstra algorithm and the user-facing web interface.

Technical Commentary

Data Extraction: The system uses request.get_json() to handle asynchronous (AJAX) data sent from the frontend.

Algorithmic Integration: It triggers the find_shortest_path method, which is the heart of the system's utility.

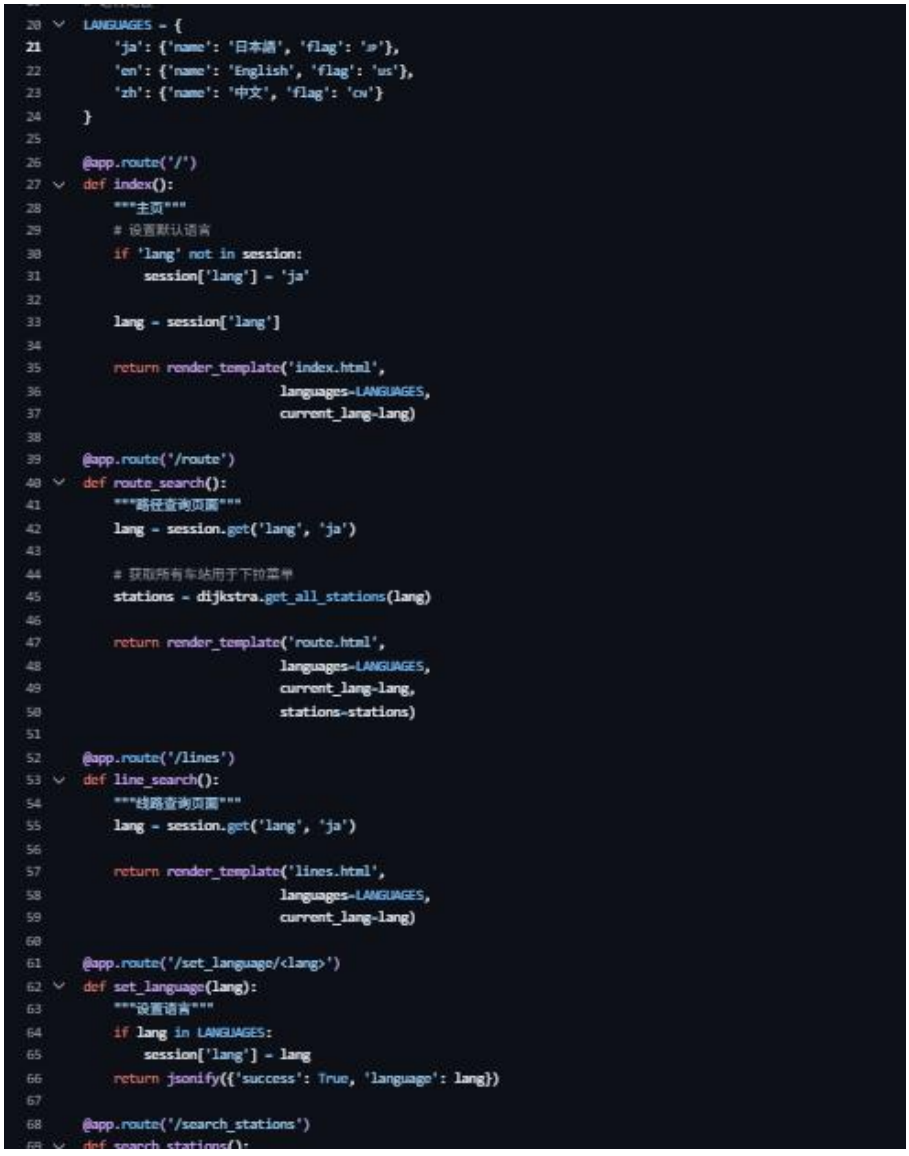
Temporal Precision: By using pytz, the system ensures that "Arrival Time" is always calculated based on actual Tokyo time, regardless of where the server is hosted.



```python
def calculate_route():
    """计算最短路径"""
    data = request.get_json()

    if not data:
        return jsonify({'error': '无效的请求数据'}), 400

    start_station = data.get('start_station')
    end_station = data.get('end_station')
    lang = session.get('lang', 'ja')

    if not start_station or not end_station:
        return jsonify({'error': '请选择起始站和终点站'}), 400

    # 计算最短路径
    path, total_time = dijkstra.find_shortest_path(start_station, end_station)

    if not path:
        return jsonify({'error': '未找到从起始站到终点站的路径'}), 404

    # 获取路径详情
    path_details = dijkstra.get_path_details(path, lang)

    # 计算时间信息
    from datetime import datetime, timedelta
    import pytz

    # 获取当前东京时间
    tokyo_tz = pytz.timezone('Asia/Tokyo')
    current_time = datetime.now(tokyo_tz)

    # 计算出发时间和到达时间
    departure_time = current_time
    arrival_time = current_time + timedelta(minutes=total_time)

    # 格式化时间显示
    time_format = {
        'ja': '%Y年%m月%d日 %H時%M分',
        'en': '%Y-%m-%d %H:%M',
        'zh': '%Y年%m月%d日 %H时%M分'
    }.get(lang, '%Y-%m-%d %H:%M')

    # 准备响应数据
    result = {
        'success': True,
        'total_time': total_time,
        'path_details': path_details,
        'start_station': get_station_name(start_station, lang),
        'end_station': get_station_name(end_station, lang),
        'time_info': {
            'departure_time': departure_time.strftime(time_format),
            'arrival_time': arrival_time.strftime(time_format),
            'current_time': current_time.strftime(time_format),
            'travel_time': total_time
        }
    }

    return jsonify(result)
```

Figure 5.1 The Route Calculation Engine

Figure 5.2 The application ensures a consistent user experience by utilizing Flask's session management to persist language preferences throughout the browsing session. Within the index and set_language routes, the system checks for the presence of a 'lang' key in the session; if absent, it defaults to Japanese. When a user switches languages via the interface, the set_language endpoint updates the session data and returns a JSON confirmation, allowing the entire application—from station names in the dropdowns to the descriptive text in the "About" section—to adapt immediately to the user's linguistic choice without losing their current context.



```
LANGUAGES = {
    'ja': {'name': '日本語', 'flag': 'jp'},
    'en': {'name': 'English', 'flag': 'us'},
    'zh': {'name': '中文', 'flag': 'cn'}
}

@app.route('/')
def index():
    """主页"""
    # 设置默认语言
    if 'lang' not in session:
        session['lang'] = 'ja'

    lang = session['lang']

    return render_template('index.html',
                           languages=LANGUAGES,
                           current_lang=lang)

@app.route('/route')
def route_search():
    """路径查询页面"""
    lang = session.get('lang', 'ja')

    # 获取所有车站用于下拉菜单
    stations = dijkstra.get_all_stations(lang)

    return render_template('route.html',
                           languages=LANGUAGES,
                           current_lang=lang,
                           stations=stations)

@app.route('/lines')
def line_search():
    """线路查询页面"""
    lang = session.get('lang', 'ja')

    return render_template('lines.html',
                           languages=LANGUAGES,
                           current_lang=lang)

@app.route('/set_language/<lang>')
def set_language(lang):
    """设置语言"""
    if lang in LANGUAGES:
        session['lang'] = lang
    return jsonify({'success': True, 'language': lang})

@app.route('/search_stations')
def search_stations():
```

Figure 5.2 Language and Session Persistence

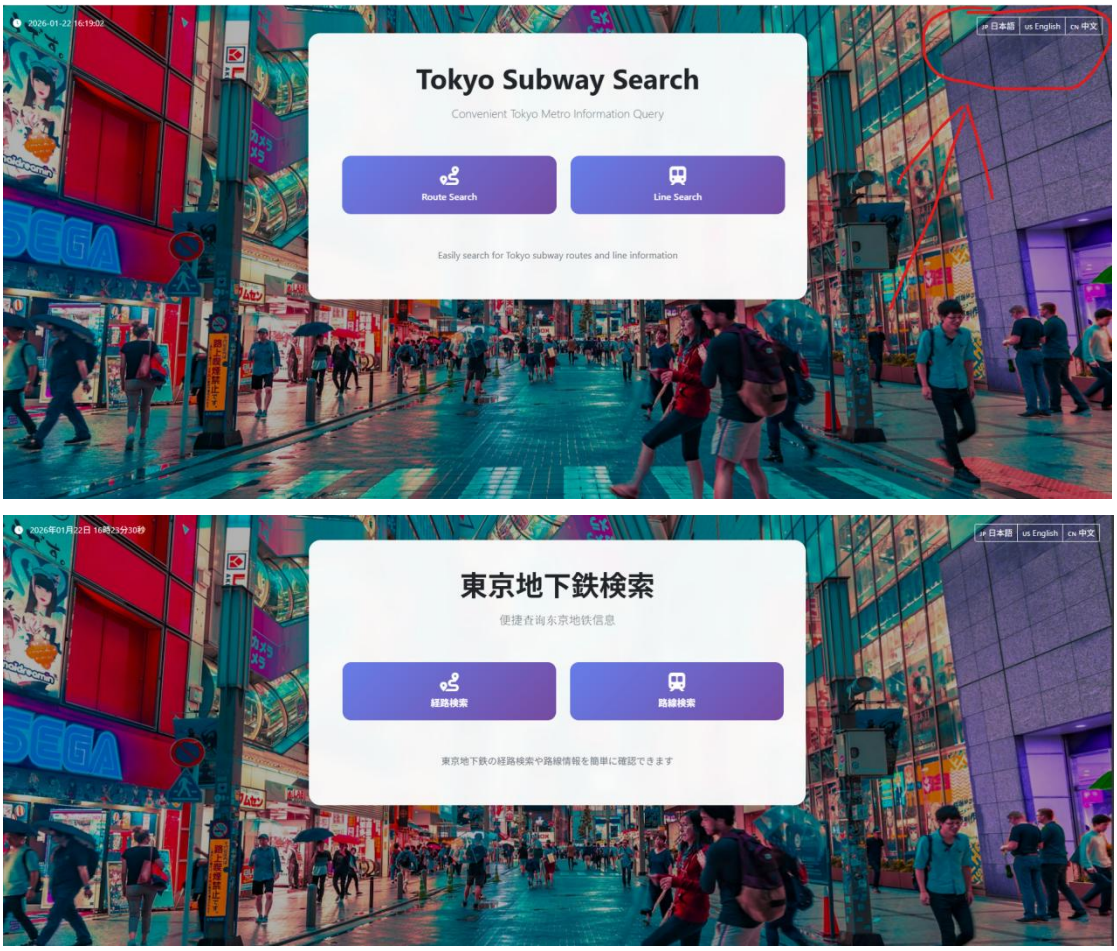


Figure 5.3 Language and conversational persistence user interface

Figure 5.4 To enhance the efficiency of the user interface, the search_stations and line_stations endpoints provide filtered, organized data that simplifies navigation through the complex Tokyo network. The search functionality performs real-time filtering based on user input, while the line-specific route ensures that stations are not merely listed alphabetically but are returned in their actual operational sequence by calling get_line_station_order. This logic allows the frontend to render a logical progression of the subway line, providing the user with a more intuitive understanding of the geographic and operational flow of the transit system.

```python
def line_stations(line_code):
    """获取特定线路的车站列表"""
    lang = session.get('lang', 'ja')

    # 获取线路的车站运行顺序
    from tokyo_subway_data import get_line_station_order
    station_order = get_line_station_order(line_code)

    stations = []

    # 按运行顺序排列车站
    for station_id in station_order:
        station_lines = get_station_lines(station_id)
        if line_code in station_lines:
            station_name = get_station_name(station_id, lang)
            stations.append({
                'id': station_id,
                'name': station_name,
                'lines': station_lines,
                'order': len(stations) + 1  # 添加顺序编号
            })

    # 如果没有找到顺序，按名称排序作为备用
    if not stations:
        from tokyo_subway_data import TOKYO_SUBWAY_STATIONS
        for station_id in TOKYO_SUBWAY_STATIONS['ja'].keys():
            station_lines = get_station_lines(station_id)
            if line_code in station_lines:
                station_name = get_station_name(station_id, lang)
                stations.append({
                    'id': station_id,
                    'name': station_name,
                    'lines': station_lines,
                    'order': len(stations) + 1
                })
        stations.sort(key=lambda x: x['name'])

    return jsonify(stations)
```

Figure 5.4 Intelligent Station Search and Line Ordering

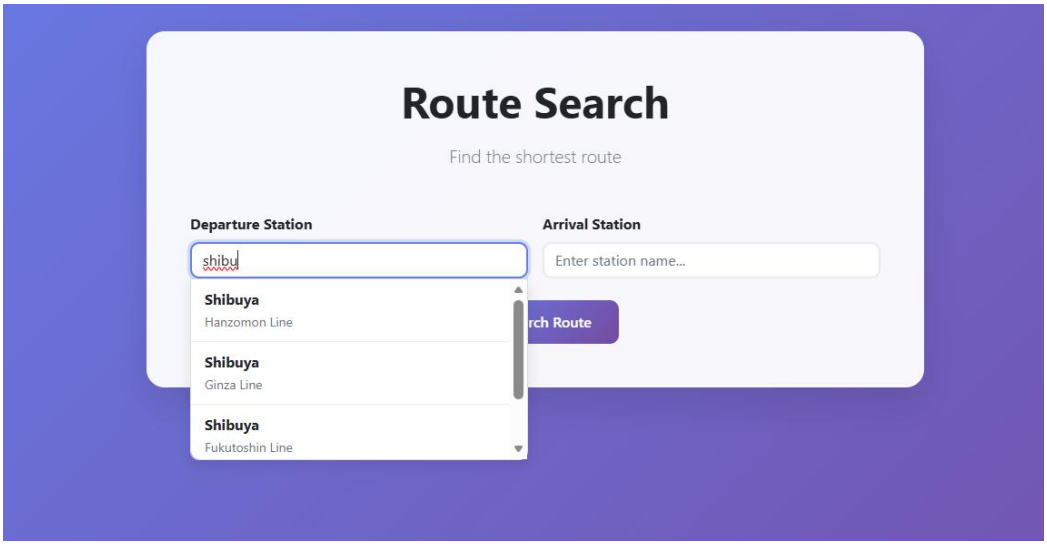


Figure 5.5 Station Search and Line User Interface

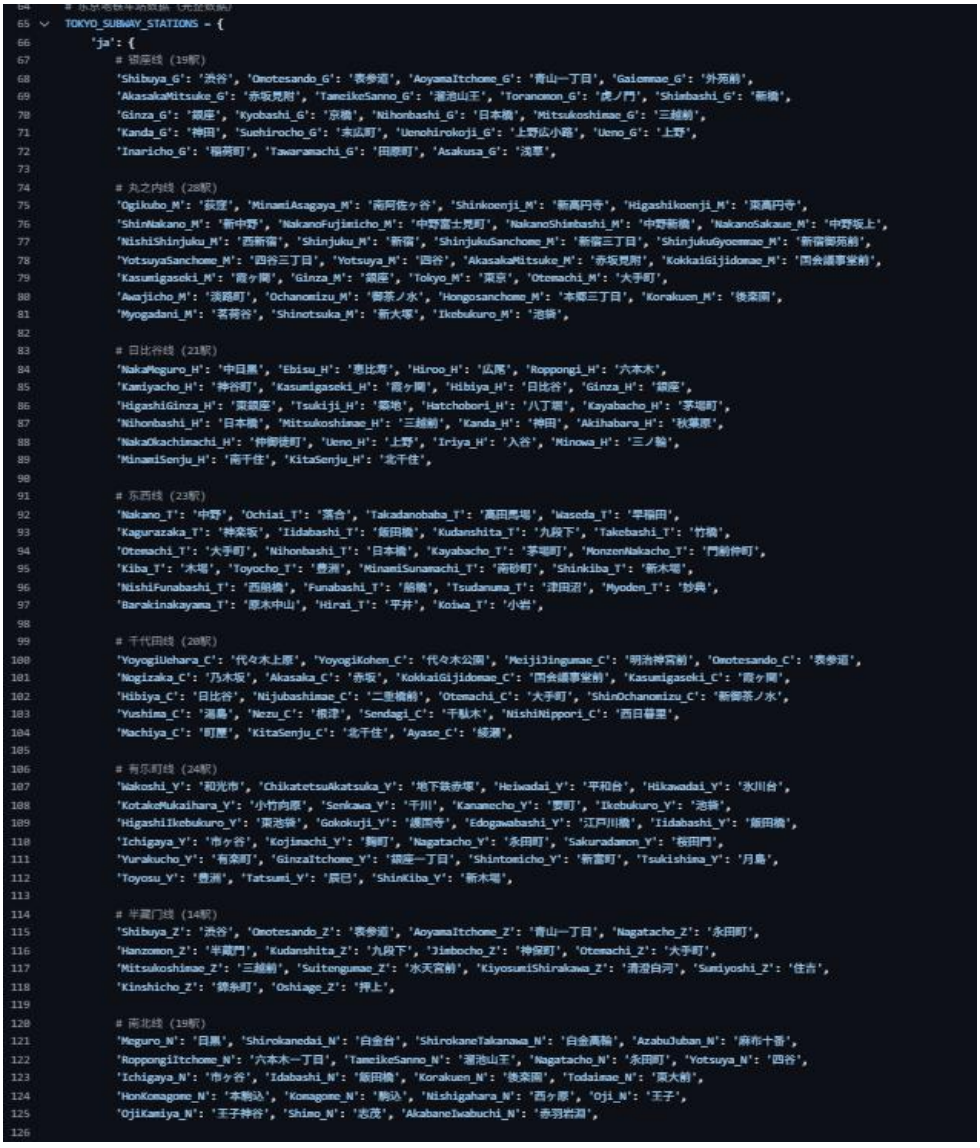
Figure 5.6 The application maintains professional robustness through centralized error handling routes that intercept standard HTTP errors like 404 and 500. Instead of displaying generic browser error messages, these handlers actively retrieve the user's current language preference from the session to deliver a localized explanation of the issue. This ensures that even when a user encounters a broken link or a server-side exception, the system remains communicative and supportive, maintaining the "Tokyo Subway" branding and navigation elements to help the user return to a functional state.

```
266  def not_found(error):
267      lang = session.get('lang', 'ja')
268
269      error_messages = {
270          'ja': 'ページが見つかりません',
271          'en': 'Page not found',
272          'zh': '页面未找到'
273      }
274
275      return render_template('error.html',
276                             error_message=error_messages.get(lang, 'Page not found'),
277                             languages=LANGUAGES,
278                             current_lang=lang), 404
279
280  @app.errorhandler(500)
281  def internal_error(error):
282      lang = session.get('lang', 'ja')
283
284      error_messages = {
285          'ja': '内部サーバーエラー',
286          'en': 'Internal server error',
287          'zh': '内部服务器错误'
288      }
289
290      return render_template('error.html',
291                             error_message=error_messages.get(lang, 'Internal server error'),
292                             languages=LANGUAGES,
293                             current_lang=lang), 500
294
295  @app.route('/export_panorama')
```

Figure 5.6 Global Error Management and Localization

tokyo_metro_data

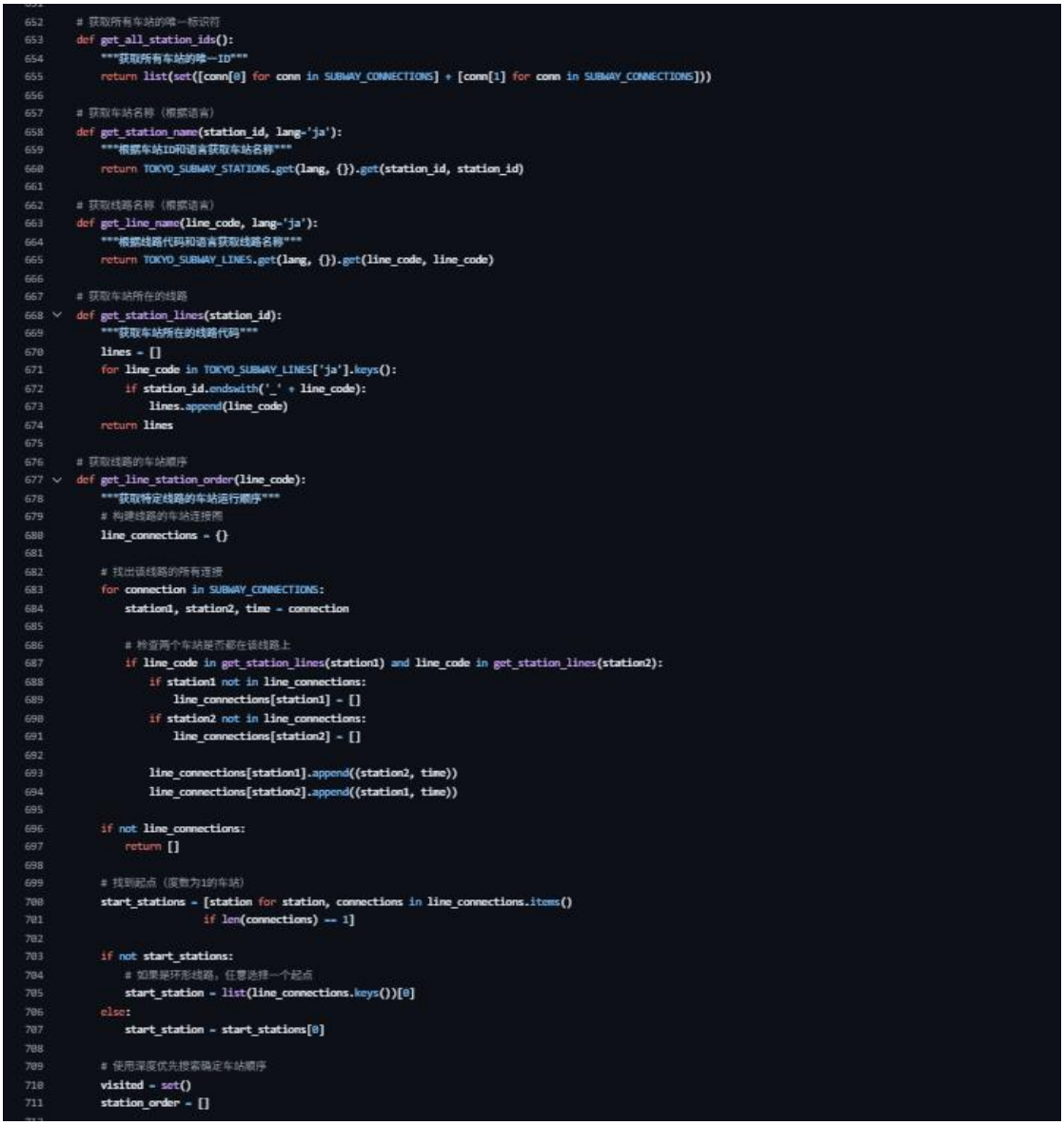
Figure 5.7 The foundation of the system's data layer is a structured dictionary that serves as a multi-language repository for all subway stations and lines. This module maps unique station identifiers to their respective names in Japanese, English, and Chinese, ensuring that the application can provide a localized experience without redundant logic. By centralizing this metadata, the system maintains a "single source of truth," allowing other components like the Dijkstra algorithm or the Flask controller to retrieve consistent naming conventions across the entire Tokyo subway network regardless of the user's interface language.



```
TOKYO_SUBWAY_STATIONS = {
    'ja': {
        # 银座线 (19駅)
        'Shibuya_G': '渋谷', 'Omotesando_G': '表参道', 'AoyamaItchome_G': '青山一丁目', 'Gaiemmae_G': '外苑前',
        'AkasakaMitsuke_G': '赤坂見附', 'TameikeSanno_G': '溜池山王', 'Toranomon_G': '虎ノ門', 'Shimbashi_G': '新橋',
        'Ginza_G': '銀座', 'Kyobashi_G': '京橋', 'Nihonbashi_G': '日本橋', 'Mitsukoshimae_G': '三越前',
        'Kanda_G': '神田', 'Suehirocho_G': '末広町', 'Ueno-hirokoji_G': '上野広小路', 'Ueno_G': '上野',
        'Inaricho_G': '稲荷町', 'Tawaramachi_G': '田原町', 'Asakusa_G': '浅草',

        # 丸之内线 (28駅)
        'Ogikubo_M': '荻窪', 'MinamiAsagaya_M': '南阿佐ケ谷', 'Shinkoenji_M': '新高円寺', 'Higashikoenji_M': '東高円寺',
        'ShinNakano_M': '新中野', 'NakanoFujimicho_M': '中野富士見町', 'NakanoShimbashi_M': '中野新橋', 'NakanoSakaue_M': '中野坂上',
        'NishiShinjuku_M': '西新宿', 'Shinjuku_M': '新宿', 'ShinjukuSanchome_M': '新宿三丁目', 'ShinjukuGyoemmae_M': '新宿御苑前',
        'YotsuyaSanchome_M': '四谷三丁目', 'Yotsuya_M': '四谷', 'AkasakaMitsuke_M': '赤坂見附', 'KokkaiGijidomae_M': '国会議事堂前',
        'Kasumigaseki_M': '霞ヶ関', 'Ginza_M': '銀座', 'Tokyo_M': '東京', 'Otemachi_M': '大手町',
        'Awajicho_M': '淡路町', 'Ochanomizu_M': '御茶ノ水', 'Hongosanchome_M': '本郷三丁目', 'Korakuen_M': '後楽園',
        'Myogadani_M': '茗荷谷', 'Shinotsuka_M': '新大塚', 'Ikebukuro_M': '池袋',

        # 日比谷线 (21駅)
        'NakaMeguro_H': '中目黒', 'Ebisu_H': '恵比寿', 'Hiroo_H': '広尾', 'Roppongi_H': '六本木',
        'Kamiyacho_H': '神谷町', 'Kasumigaseki_H': '霞ヶ関', 'Hibiya_H': '日比谷', 'Ginza_H': '銀座',
        'HigashiGinza_H': '東銀座', 'Tsukiji_H': '築地', 'Hatchobori_H': '八丁堀', 'Kayabacho_H': '茅場町',
        'Nihonbashi_H': '日本橋', 'Mitsukoshimae_H': '三越前', 'Kanda_H': '神田', 'Akihabara_H': '秋葉原',
        'NakaOkachimachi_H': '仲御徒町', 'Ueno_H': '上野', 'Iriya_H': '入谷', 'Minowa_H': '三ノ輪',
        'MinamiSenju_H': '南千住', 'KitaSenju_H': '北千住',

        # 东西线 (23駅)
        'Nakano_T': '中野', 'Ochiai_T': '落合', 'Takadanobaba_T': '高田馬場', 'Waseda_T': '早稲田',
        'Kagurazaka_T': '神楽坂', 'Iidabashi_T': '飯田橋', 'Kudanshita_T': '九段下', 'Takebashi_T': '竹橋',
        'Otemachi_T': '大手町', 'Nihonbashi_T': '日本橋', 'Kayabacho_T': '茅場町', 'MonzenNakacho_T': '門前仲町',
        'Kiba_T': '木場', 'Toyocho_T': '豊洲', 'MinamiSunamachi_T': '南砂町', 'Shinkiba_T': '新木場',
        'NishiFunabashi_T': '西船橋', 'Funabashi_T': '船橋', 'Tsudanuma_T': '津田沼', 'Myoden_T': '妙典',
        'Baraki-nakayama_T': '原木中山', 'Hirai_T': '平井', 'Koiwa_T': '小岩',

        # 千代田线 (20駅)
        'YoyogiUehara_C': '代々木上原', 'YoyogiKohen_C': '代々木公園', 'MeijiJingumae_C': '明治神宮前', 'Omotesando_C': '表参道',
        'Nogizaka_C': '乃木坂', 'Akasaka_C': '赤坂', 'KokkaiGijidomae_C': '国会議事堂前', 'Kasumigaseki_C': '霞ヶ関',
        'Hibiya_C': '日比谷', 'Nijubashimae_C': '二重橋前', 'Otemachi_C': '大手町', 'ShinOchanomizu_C': '新御茶ノ水',
        'Yushima_C': '湯島', 'Nezu_C': '根津', 'Sendagi_C': '千駄木', 'NishiNippori_C': '西日暮里',
        'Machiya_C': '町屋', 'KitaSenju_C': '北千住', 'Ayase_C': '綾瀬',

        # 有乐町线 (24駅)
        'Wakoshi_Y': '和光市', 'ChikatetsuAkatsuka_Y': '地下鉄赤塚', 'Heiwadai_Y': '平和台', 'Hikawadai_Y': '氷川台',
        'KotakeMukaihara_Y': '小竹向原', 'Senkawa_Y': '千川', 'Kanamecho_Y': '要町', 'Ikebukuro_Y': '池袋',
        'HigashiIkebukuro_Y': '東池袋', 'Gokokuji_Y': '護国寺', 'Edogawabashi_Y': '江戸川橋', 'Iidabashi_Y': '飯田橋',
        'Ichigaya_Y': '市ヶ谷', 'Kojimachi_Y': '麹町', 'Nagatacho_Y': '永田町', 'Sakuradamon_Y': '桜田門',
        'Yurakucho_Y': '有楽町', 'GinzaItchome_Y': '銀座一丁目', 'Shintomicho_Y': '新富町', 'Tsukishima_Y': '月島',
        'Toyosu_Y': '豊洲', 'Tatsumi_Y': '辰巳', 'ShinKiba_Y': '新木場',

        # 半藏门线 (14駅)
        'Shibuya_Z': '渋谷', 'Omotesando_Z': '表参道', 'AoyamaItchome_Z': '青山一丁目', 'Nagatacho_Z': '永田町',
        'Hanzomon_Z': '半蔵門', 'Kudanshita_Z': '九段下', 'Jimbocho_Z': '神保町', 'Otemachi_Z': '大手町',
        'Mitsukoshimae_Z': '三越前', 'Suitengumae_Z': '水天宮前', 'KiyosumiShirakawa_Z': '清澄白河', 'Sumiyoshi_Z': '住吉',
        'Kinshicho_Z': '錦糸町', 'Oshiage_Z': '押上',

        # 南北线 (19駅)
        'Meguro_N': '目黒', 'Shirokanedai_N': '白金台', 'ShirokaneTakanawa_N': '白金高輪', 'AzabuJuban_N': '麻布十番',
        'Roppongiitchome_N': '六本木一丁目', 'TameikeSanno_N': '溜池山王', 'Nagatacho_N': '永田町', 'Yotsuya_N': '四谷',
        'Ichigaya_N': '市ヶ谷', 'Idabashi_N': '飯田橋', 'Korakuen_N': '後楽園', 'Todaimae_N': '東大前',
        'HonKomagome_N': '本駒込', 'Komagome_N': '駒込', 'Nishigahara_N': '西ヶ原', 'Oji_N': '王子',
        'OjiKamiya_N': '王子神谷', 'Shimo_N': '志茂', 'AkabaneIwabuchi_N': '赤羽岩淵',
```

Figure 5.7 Centralized Station and Line Metadata Management

Figure 5.8 Beyond simple naming, this file defines the topological structure of the subway system by mapping line codes to their constituent stations in their actual operational order. The get_line_station_order function is critical for rendering route maps and station lists, as it ensures that the data follows the physical sequence of the tracks rather than a random or alphabetical arrangement. Additionally, the module tracks which lines serve each specific station, enabling the system to identify transfer points—a vital component for the pathfinding algorithm to calculate transitions between different subway operators and lines.

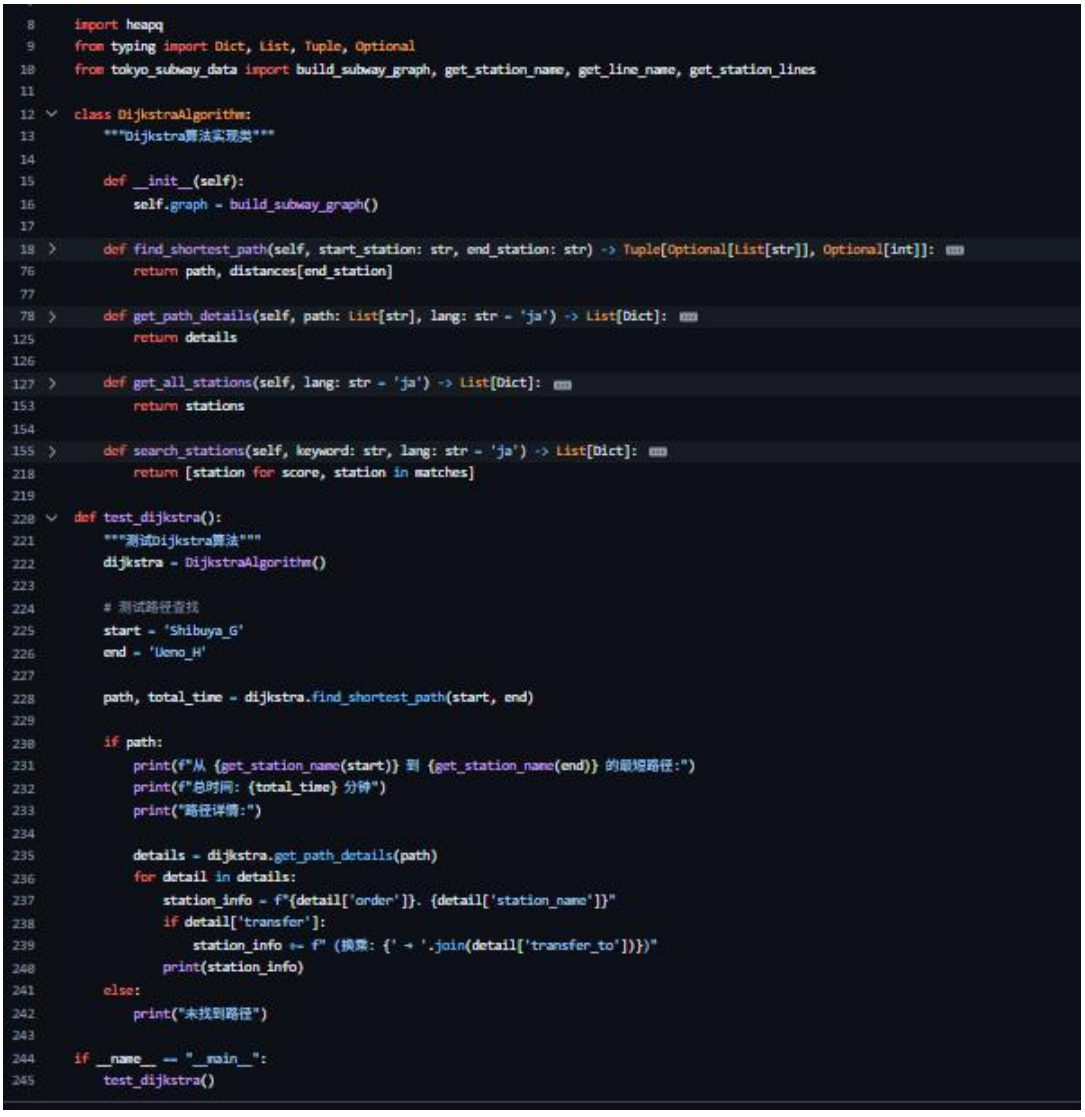


```
# 获取所有车站的唯一标识符
def get_all_station_ids():
    """获取所有车站的唯一ID"""
    return list(set([conn[0] for conn in SUBWAY_CONNECTIONS] + [conn[1] for conn in SUBWAY_CONNECTIONS]))

# 获取车站名称（根据语言）
def get_station_name(station_id, lang='ja'):
    """根据车站ID和语言获取车站名称"""
    return TOKYO_SUBWAY_STATIONS.get(lang, {}).get(station_id, station_id)

# 获取线路名称（根据语言）
def get_line_name(line_code, lang='ja'):
    """根据线路代码和语言获取线路名称"""
    return TOKYO_SUBWAY_LINES.get(lang, {}).get(line_code, line_code)

# 获取车站所在的线路
def get_station_lines(station_id):
    """获取车站所在的线路代码"""
    lines = []
    for line_code in TOKYO_SUBWAY_LINES['ja'].keys():
        if station_id.endswith('_' + line_code):
            lines.append(line_code)
    return lines

# 获取线路的车站顺序
def get_line_station_order(line_code):
    """获取特定线路的车站运行顺序"""
    # 构建线路的车站连接图
    line_connections = {}

    # 找出该线路的所有连接
    for connection in SUBWAY_CONNECTIONS:
        station1, station2, time = connection

        # 检查两个车站是否都在该线路上
        if line_code in get_station_lines(station1) and line_code in get_station_lines(station2):
            if station1 not in line_connections:
                line_connections[station1] = []
            if station2 not in line_connections:
                line_connections[station2] = []

            line_connections[station1].append((station2, time))
            line_connections[station2].append((station1, time))

    if not line_connections:
        return []

    # 找到起点（度数为1的车站）
    start_stations = [station for station, connections in line_connections.items()
                      if len(connections) == 1]

    if not start_stations:
        # 如果是环形线路，任意选择一个起点
        start_station = list(line_connections.keys())[0]
    else:
        start_station = start_stations[0]

    # 使用深度优先搜索确定车站顺序
    visited = set()
    station_order = []
```

Figure 5.8 Operational Sequencing and Line Connectivity

dijkstra_algorithm

The DijkstraAlgorithm class serves as the engine of the system, transforming static subway data into a navigable mathematical graph. Upon instantiation, the class pulls the SUBWAY_GRAPH from the data module, which contains the adjacency list of stations and the travel times between them. This initialization process ensures that the algorithm has immediate access to the entire network's topology, including transfer links—which are often modeled with a weight of zero to represent internal station connections. By encapsulating the graph within this class, the system creates a reusable computational environment where multiple pathfinding queries can be executed efficiently without re-loading the underlying data structure.



```python
import heapq
from typing import Dict, List, Tuple, Optional
from tokyo_subway_data import build_subway_graph, get_station_name, get_line_name, get_station_lines

class DijkstraAlgorithm:
    """Dijkstra算法实现类"""

    def __init__(self):
        self.graph = build_subway_graph()

    def find_shortest_path(self, start_station: str, end_station: str) -> Tuple[Optional[List[str]], Optional[int]]:
        return path, distances[end_station]

    def get_path_details(self, path: List[str], lang: str = 'ja') -> List[Dict]:
        return details

    def get_all_stations(self, lang: str = 'ja') -> List[Dict]:
        return stations

    def search_stations(self, keyword: str, lang: str = 'ja') -> List[Dict]:
        return [station for score, station in matches]

def test_dijkstra():
    """测试Dijkstra算法"""
    dijkstra = DijkstraAlgorithm()

    # 测试路径查找
    start = 'Shibuya_G'
    end = 'Ueno_H'

    path, total_time = dijkstra.find_shortest_path(start, end)

    if path:
        print(f"从 {get_station_name(start)} 到 {get_station_name(end)} 的最短路径:")
        print(f"总时间: {total_time} 分钟")
        print("路径详情:")

        details = dijkstra.get_path_details(path)
        for detail in details:
            station_info = f"{detail['order']}. {detail['station_name']}"
            if detail['transfer']:
                station_info += f" (换乘: {' → '.join(detail['transfer_to'])})"
            print(station_info)
    else:
        print("未找到路径")

if __name__ == "__main__":
    test_dijkstra()
```

Figure 5.9 Graph Initialization and Data Binding

Figure 5.10 The core of the pathfinding functionality is implemented in the find_shortest_path method, which utilizes a priority queue to execute the Dijkstra algorithm. It maintains a dictionary of the shortest known distances from the starting station to all other nodes, initialized to infinity, and iteratively relaxes these distances as it explores the network. By always selecting the station with the smallest cumulative travel time from the priority queue, the algorithm guarantees the discovery of the mathematically shortest route in terms of time. The method also tracks "predecessor" nodes, which allows the system to reconstruct the full sequence of stations from the destination back to the origin once the search is complete.

```python
def find_shortest_path(self, start_station: str, end_station: str) -> Tuple[Optional[List[str]], Optional[int]]:
    """
    使用Dijkstra算法查找最短路径

    Args:
        start_station: 起始车站ID
        end_station: 终点车站ID

    Returns:
        Tuple[路径列表, 总时间] 或 (None, None) 如果路径不存在
    """
    # 验证车站是否存在
    if start_station not in self.graph or end_station not in self.graph:
        return None, None

    # 初始化距离字典
    distances = {station: float('inf') for station in self.graph}
    distances[start_station] = 0

    # 初始化前驱节点字典
    previous = {station: None for station in self.graph}

    # 优先队列（最小堆）
    priority_queue = [(0, start_station)]

    while priority_queue:
        current_distance, current_station = heapq.heappop(priority_queue)

        # 如果找到终点站，提前结束
        if current_station == end_station:
            break

        # 如果当前距离大于已知最短距离，跳过
        if current_distance > distances[current_station]:
            continue

        # 遍历相邻车站
        for neighbor, time in self.graph[current_station].items():
            distance = current_distance + time

            # 如果找到更短路径
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                previous[neighbor] = current_station
                heapq.heappush(priority_queue, (distance, neighbor))

    # 如果终点站不可达
    if distances[end_station] == float('inf'):
        return None, None

    # 重构路径
    path = []
    current = end_station
    while current is not None:
        path.append(current)
        current = previous[current]
    path.reverse()

    return path, distances[end_station]
```

Figure 5.10 Shortest Path Computation Logic

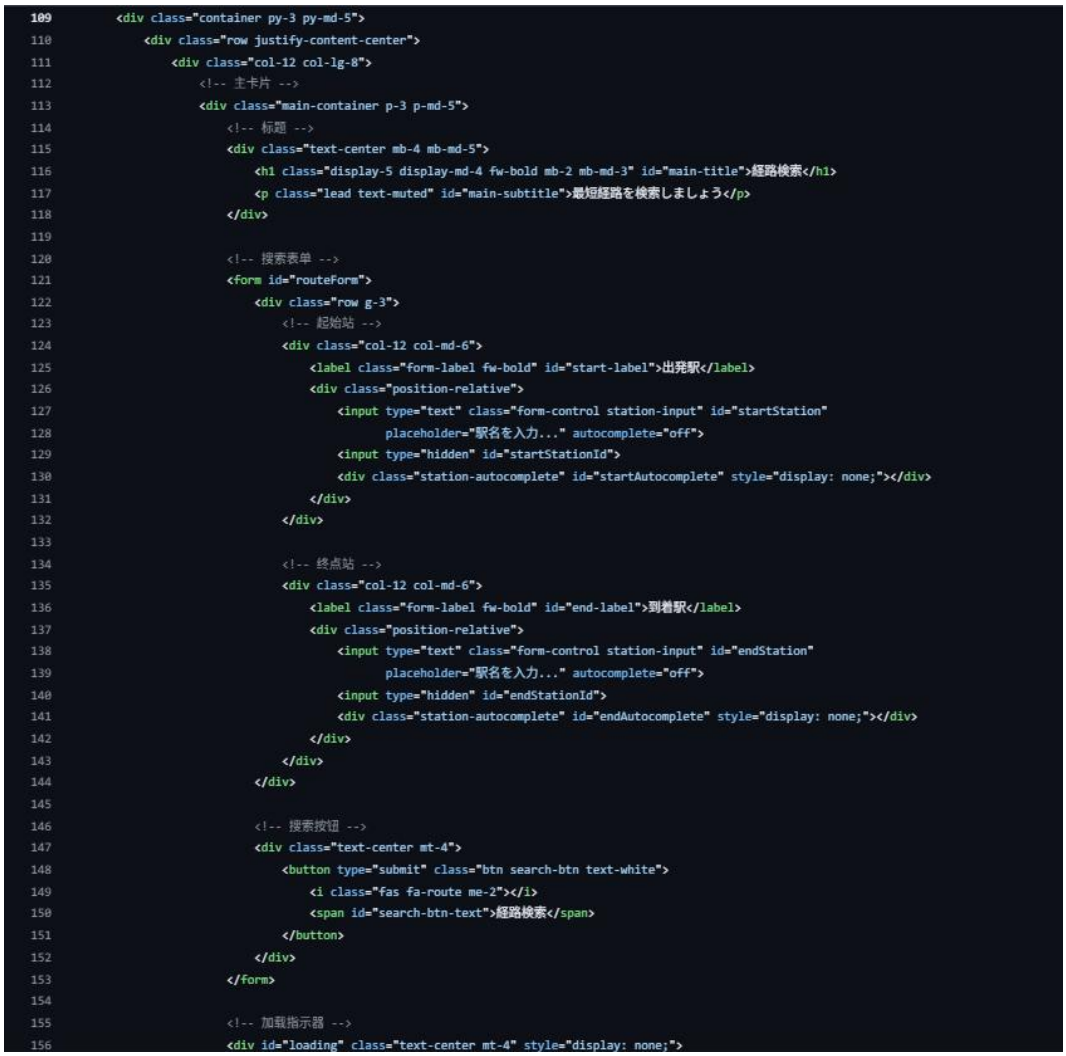
Dynamic UI Localization and State Management

In lines.html, the system showcases its ability to visualize complex data through the HTML5 <canvas> element. Rather than relying on static images, the generatePanoramaMap function dynamically draws the subway network's topological structure based on line-specific color configurations. This programmatic drawing logic allows the system to highlight specific lines, such as the circular Oedo Line or the intersecting Ginza Line, while maintaining high visual fidelity on any screen size. By offloading this rendering to the client's browser, the application provides a lightweight yet powerful visual aid that helps users understand the spatial relationships between different subway routes.

```
302         // 更新界面文本
303         function updateTexts() {
304             const texts = translations[currentLang];
305
306             document.getElementById('back-text').textContent = texts['back-text'];
307             document.getElementById('main-title').textContent = texts['main-title'];
308             document.getElementById('main-subtitle').textContent = texts['main-subtitle'];
309             document.getElementById('line-select-title').textContent = texts['line-select-title'];
310             document.getElementById('line-detail-title').textContent = texts['line-detail-title'];
311             document.getElementById('placeholder-text').textContent = texts['placeholder-text'];
312             document.getElementById('station-title').textContent = texts['station-title'];
313
314             // 更新全景图相关文本
315             const panoramaTitle = document.getElementById('panorama-title');
316             const exportBtn = document.getElementById('export-pdf-btn');
317             if (panoramaTitle) {
318                 panoramaTitle.innerHTML = `<i class=\"fas fa-map me-2\"></i>${texts['panorama-title'] || '東京地下鉄全景図'}`;
319             }
320             if (exportBtn) {
321                 exportBtn.innerHTML = `<i class=\"fas fa-file-pdf me-2\"></i>${texts['export-pdf'] || 'PDFで保存'}`;
322             }
323         }
324
325         // 生成线路列表
326         function generateLineList() {
327             const container = document.getElementById('lines-container');
328             container.innerHTML = '';
329
330             for (const [lineCode, lineInfo] of Object.entries(lineData)) {
331                 const lineItem = document.createElement('div');
332                 lineItem.className = 'line-item';
333                 lineItem.style.backgroundColor = lineInfo.color;
334                 lineItem.innerHTML = `
335                     <div class="d-flex justify-content-between align-items-center">
336                         <span class="fw-bold">${lineCode}線</span>
337                         <small>${lineInfo.stations}駅</small>
338                     </div>
339                     <small>${lineInfo.name}</small>
340                 `;
341
342                 lineItem.addEventListener('click', () => {
343                     showLineDetails(lineCode, lineInfo);
344                 });
345
346                 container.appendChild(lineItem);
347             }
348         }
```

Figure 5.11 Client-Side Map Rendering via Canvas

The route.html file implements a modern, "search-as-you-type" interface that significantly reduces user friction when navigating the massive Tokyo station list. By utilizing the fetch API, the frontend communicates asynchronously with the /search_stations backend route, retrieving matching results in real-time. The setupAutocomplete logic then renders these results in a custom dropdown, allowing users to select stations by their Japanese kanji, English romaji, or Chinese characters. This decoupled approach—where the frontend manages the UI state and the backend handles the heavy filtering—ensures the application remains responsive even when handling hundreds of station data points.



```
109     <div class="container py-3 py-md-5">
110         <div class="row justify-content-center">
111             <div class="col-12 col-lg-8">
112                 <!-- 主卡片 -->
113                 <div class="main-container p-3 p-md-5">
114                     <!-- 标题 -->
115                     <div class="text-center mb-4 mb-md-5">
116                         <h1 class="display-5 display-md-4 fw-bold mb-2 mb-md-3" id="main-title">経路検索</h1>
117                         <p class="lead text-muted" id="main-subtitle">最短経路を検索しましょう</p>
118                     </div>
119
120                     <!-- 搜索表单 -->
121                     <form id="routeForm">
122                         <div class="row g-3">
123                             <!-- 起始站 -->
124                             <div class="col-12 col-md-6">
125                                 <label class="form-label fw-bold" id="start-label">出発駅</label>
126                                 <div class="position-relative">
127                                     <input type="text" class="form-control station-input" id="startStation"
128                                            placeholder="駅名を入力..." autocomplete="off">
129                                     <input type="hidden" id="startStationId">
130                                     <div class="station-autocomplete" id="startAutocomplete" style="display: none;"></div>
131                                 </div>
132                             </div>
133
134                             <!-- 终点站 -->
135                             <div class="col-12 col-md-6">
136                                 <label class="form-label fw-bold" id="end-label">到着駅</label>
137                                 <div class="position-relative">
138                                     <input type="text" class="form-control station-input" id="endStation"
139                                            placeholder="駅名を入力..." autocomplete="off">
140                                     <input type="hidden" id="endStationId">
141                                     <div class="station-autocomplete" id="endAutocomplete" style="display: none;"></div>
142                                 </div>
143                             </div>
144                         </div>
145
146                         <!-- 搜索按钮 -->
147                         <div class="text-center mt-4">
148                             <button type="submit" class="btn search-btn text-white">
149                                 <i class="fas fa-route me-2"></i>
150                                 <span id="search-btn-text">経路検索</span>
151                             </button>
152                         </div>
153                     </form>
154
155                     <!-- 加载指示器 -->
156                     <div id="loading" class="text-center mt-4" style="display: none;">
```

Figure 5.12 Interactive Asynchronous Search and Autocomplete

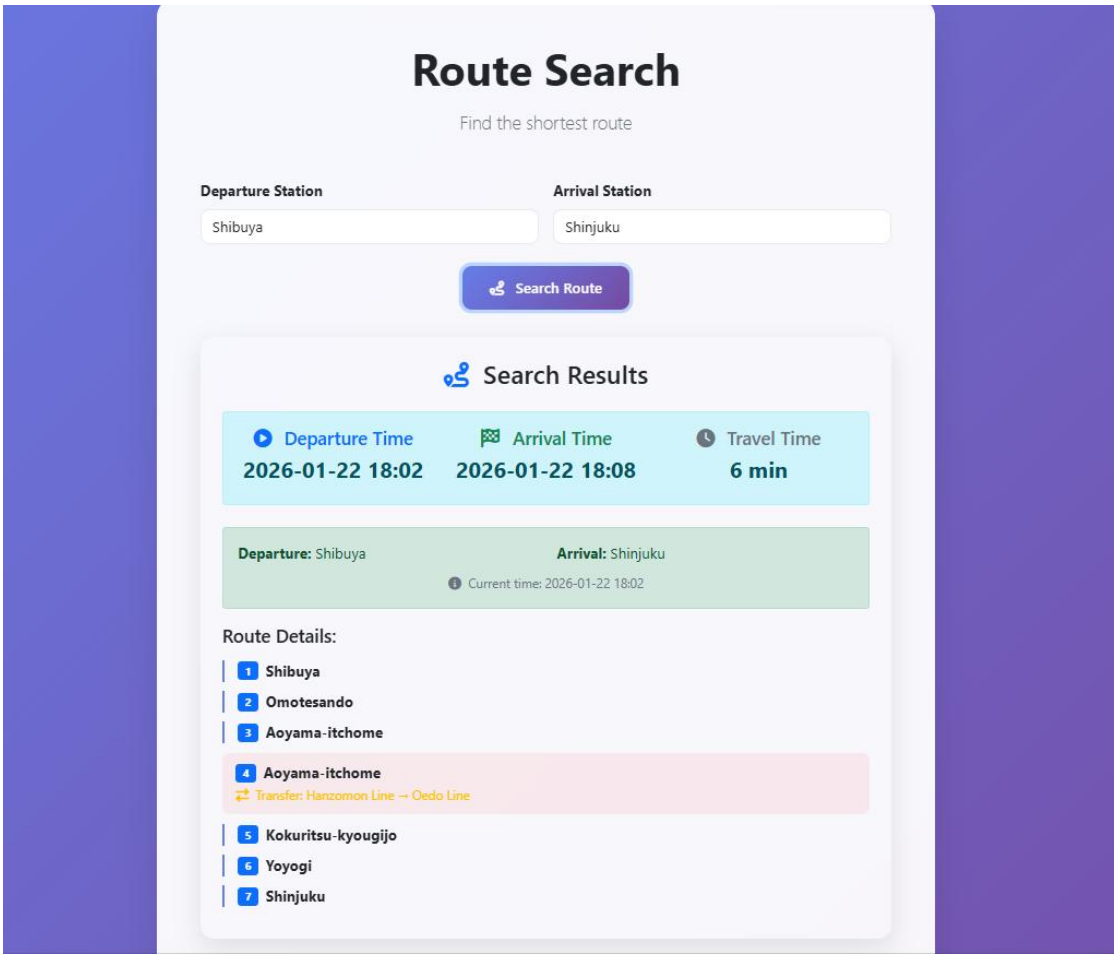


Figure 5.13 Advanced Route Search and Result Visualization

The about.html and error.html templates demonstrate how the system maintains a professional and consistent brand identity even on secondary or fallback pages. These files utilize a unified CSS background gradient and a glassmorphism-style container to create a modern aesthetic. The critical logic here is the updateAllTexts() function, which acts as a client-side localization engine; by iterating through DOM elements with specific data-lang attributes, it ensures that project descriptions and error messages ("Page Not Found") are instantly translated without a server round-trip, providing a seamless experience for international travelers.

The route.html file is the most interactive component of the frontend, managing complex form states and asynchronous data fetching. It implements a dual-layer input system where the user sees a friendly station name while a hidden field stores the precise station ID for the backend. Upon a successful route calculation, the page dynamically injects a detailed "Route Result" card. This card uses Font Awesome icons and localized time calculations to present the journey in a vertical timeline format, clearly marking transfer points and estimated arrival times, which transforms abstract algorithmic data into actionable travel instructions.

5.2.3 Important Modules

The proposed system is implemented using a modular file-based structure, where each source file represents an important functional module of the system. This design approach improves code organization, readability, and maintainability by clearly separating different system responsibilities. The important modules are explained based on their corresponding source files as follows.

app.py – Application Control and Routing Module

The app.py file serves as the main application controller of the system. It is responsible for handling incoming user requests through defined web routes and coordinating the overall system workflow.

This module receives input data from the frontend interface, performs basic input validation, and determines which backend functions and algorithms should be executed. It acts as the central communication hub between the user interface, data processing module, and algorithm module, ensuring that the system operates in a structured and controlled manner.

dijkstra_algorithm.py – Core Algorithm Module

The dijkstra_algorithm.py file implements the core routing algorithm used in the system. This module contains the logic required to compute the shortest path between stations based on the provided transportation network data.

As the most critical computational module, this file directly affects the accuracy and efficiency of the system's output. The separation of the algorithm into an independent module allows the system to be easily extended or improved by modifying the algorithm without affecting other components.

tokyo_subway_data.py – Data Processing and Preparation Module

The tokyo_subway_data.py file is responsible for handling system data related to the Tokyo subway network. This includes loading raw data, organizing station and connection information, and constructing the graph structure required by the routing algorithm.

This module ensures that data is consistently formatted and correctly structured before being used by the algorithm module. Proper data preparation in this stage is essential for ensuring reliable and accurate route calculations.

HTML Template Files – User Interface Module

The HTML template files represent the frontend user interface module of the system. These files define the layout and structure of the web interface, allowing users to input required parameters and view system results.

The user interface module focuses on clarity and ease of use, ensuring smooth interaction between users and the system. It communicates with the backend through form submissions and displays system responses generated by the backend logic.

Supporting Configuration and Utility Files

In addition to the main modules, the system includes supporting files that assist with configuration and execution, such as dependency configuration files. These files support the development and deployment process but do not directly affect core system logic.

Such separation ensures that critical system functionalities remain isolated from configuration and auxiliary concerns, improving overall system organization.

5.2.4 Important Components

The system is composed of several important components, each contributing to the overall functionality and performance. Unlike modules which refer to individual files, components represent logical building blocks of the system, often spanning multiple files or modules. The key components of the proposed system are described as follows.

Frontend User Interface Component

This component is responsible for all interactions with the user. It provides an intuitive web-based interface where users can input the starting station, destination station, and any additional preferences, and then view the calculated route results.

Key responsibilities:

Display forms for user input

Present system output in a clear and readable format

Facilitate communication with the backend server through HTTP requests

Files involved:

HTML template files (index.html, result.html, etc.)

CSS and JavaScript files supporting interactivity

This component ensures ease of use and accessibility, allowing users to interact with the system without requiring technical knowledge.

Backend Server Component

The backend server acts as the core processing engine of the system. It receives input from the frontend, validates data, and coordinates calls to the data processing and algorithm components.

Key responsibilities:

Handle HTTP requests and responses (app.py)

Manage system workflow and routing logic

Ensure data is processed correctly before passing to algorithms

Return results to the frontend for display

This component is critical for system reliability and workflow coordination.

Algorithm Component

This component implements the route calculation logic, using the shortest-path algorithm to determine optimal travel routes.

Key responsibilities:

Execute Dijkstra's shortest path algorithm (dijkstra_algorithm.py)

Compute optimal paths between stations

Handle route constraints and special cases

By isolating the algorithm in its own component, the system allows future improvements or replacement of the algorithm without affecting other components.

Data Management Component

This component is responsible for storing, organizing, and preparing data for processing. It handles the structured information of the Tokyo subway network, including station data, connection data, and travel distances.

Key responsibilities:

Load data from CSV or database (tokyo_subway_data.py)

Construct graph structures used by the algorithm

Validate and clean data before processing

Error Handling and Validation Component

This component ensures the system can handle exceptional situations gracefully, preventing crashes or incorrect outputs.

Key responsibilities:

Validate user input

Detect and handle missing or invalid data

Provide informative feedback to users in case of errors

This component enhances system robustness and ensures a reliable user experience.

5.2.5 Important libraries

Several third-party libraries were used to support the development of the system:

Flask – A lightweight web framework used for backend routing, request handling, and integration with frontend templates. It provides flexibility and rapid development capabilities.

Pandas – Handles data loading, cleaning, and organization. Ensures structured and accurate data for algorithm processing.

NumPy – Supports numerical operations and efficient computations, essential for algorithm calculations.

Matplotlib / Plotting Libraries – Used for visualizing routes, graphs, and debugging outputs.

OS / System Utilities – Provides file handling and system-level support to ensure compatibility and smooth operation.

These libraries collectively enable efficient development, reliable data processing, and maintainable system implementation.

5.2.6 Database

The system uses a structured data storage approach to manage Tokyo subway network information. The database contains station names, connections, and travel distances, which are essential for route calculation.

Data Storage Format: CSV files are used to store structured data in tables for stations and connections.

Data Access: Data is loaded into the system at runtime and organized into graph structures for algorithm processing.

Purpose: Ensures accurate, consistent, and accessible data for the shortest path algorithm.

Scalability: The structure allows easy addition of new stations or routes without major system changes.

This database component is crucial for system reliability and correctness, as all computations depend on properly structured and validated data.

5.3 SUMMARY

This chapter has presented the implementation of the proposed system, detailing the development process, critical code segments, important modules and components, libraries, and database design. The implementation followed a structured and iterative approach, ensuring that all functional requirements defined in earlier stages were accurately translated into a working system.

The system is organized into modular components, each with clearly defined responsibilities. The Frontend User Interface provides intuitive interaction for users to input starting and destination stations. The Backend Server and Application Control Module coordinate system workflows and manage request processing. The Algorithm Module executes the shortest path calculation using Dijkstra’s algorithm, while the Data Management Module ensures that all subway network data is accurate, consistent, and efficiently structured. Error handling and validation mechanisms are implemented

throughout to ensure robustness and reliability. The use of key libraries such as Flask, Pandas, and NumPy facilitated efficient development, accurate computation, and smooth data manipulation.

During the development phase, several challenges were encountered, including integrating frontend and backend components, managing complex data structures for the subway network, and handling edge cases in route computation. These challenges were addressed through careful debugging, iterative refinement, and testing of each module and component.

In conclusion, the implementation phase successfully transformed the design specifications into a fully functional, maintainable, and scalable system. The modular design, combined with structured data management and robust algorithm implementation, ensures that the system meets its primary objectives. Furthermore, the architecture allows for future extensions, such as the inclusion of additional transportation data, optimization of algorithm performance, or enhancements to the user interface, making the system adaptable to evolving requirements.

CHAPTER VI

SYSTEM TESTING

6.1 INTRODUCTION

This chapter presents the testing and evaluation of the proposed system. The main purpose is to verify that the system functions correctly, meets the design specifications, and provides a reliable and user-friendly experience.

The testing activities focus on four main areas: functional correctness, interface usability, exception handling, and overall system performance. Through structured tests and systematic evaluation, potential issues are identified and addressed to ensure that the system is robust, stable, and ready for deployment.

6.2 TEST PLAN

The test plan outlines the strategy, methods, and criteria used to evaluate the system. Testing is conducted in a controlled environment, with well-defined inputs and expected outputs for each test scenario. The plan includes tests for functionality, user interface, and exception handling.

6.2.1 Testing Environment

The testing of the proposed system was conducted in a controlled environment to ensure consistent and reliable results. The testing environment is divided into hardware and software configurations as described below.

Hardware Environment

The hardware used during system testing is summarized as follows:

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent
Memory (RAM)	8 GB
Storage	256 GB SSD
Operating Device	Laptop / Desktop Computer
Network	Stable Internet connection

Table 6.1 Hardware Environment

The selected hardware configuration is sufficient to support system execution, testing activities, and performance evaluation without hardware-related limitations.

Software Environment

The software environment used for system testing includes the following:

Component	Specification
Operating System	Windows 10
Programming Language	Python 3.12
Web Framework	Flask
Web Browser	Google Chrome
Data Storage	SQLite
Development Tools	Visual Studio Code

Table 6.2 Software Environment

This software environment ensures compatibility with the system implementation and supports reliable testing across different scenarios.

6.2.2 Functional Testing

Functional testing was conducted to verify that all core functionalities of the proposed system operate according to the defined requirements. This type of testing focuses on validating system behavior by comparing expected outputs with actual outputs generated by the system under different input conditions.

The main objectives of functional testing are to ensure that:

- The system correctly accepts and processes user input.
- Core functions such as route calculation and data processing work as intended.
- The system produces accurate and consistent results.
- No unintended behavior occurs during normal system operation.

Functional testing was performed based on predefined test cases that cover normal operations, boundary conditions, and invalid input scenarios.

Functional Test Scenarios

The following functional areas were tested:

User Input Processing

Verifies that the system correctly accepts starting and destination stations and processes the input without errors.

Route Calculation Functionality

Ensures that the shortest route is calculated accurately based on the subway network data.

Data Loading and Processing

Confirms that the system correctly loads and prepares subway data before executing the algorithm.

Result Display

Verifies that calculated routes and related information are displayed correctly on the user interface.

Table 6.3 presents the results of the functional test.

Case ID	Test Module	Testing Operation	Expected Results	Conclusion
FT01	User Input Module	Enter valid starting and destination stations	System accepts input and proceeds to route calculation	Pass
FT02	Route Calculation Module	Calculate shortest route between two valid stations	Correct shortest route and total distance are displayed	Pass
FT03	Route Calculation Module	Reverse start and destination stations	Valid route is calculated in reverse direction	Pass
FT04	Data Processing Module	Load subway network data at system startup	Data is loaded successfully without errors	Pass
FT05	Result Display Module	Display calculated route on user interface	Route path and related information are shown clearly	Pass
FT06	Input Validation Module	Enter an invalid station name	System displays an appropriate error message	Pass
FT07	Input Validation Module	Submit empty input fields	System prompts user to enter required values	Pass
FT08	Algorithm Module	Execute shortest path algorithm repeatedly	Consistent and accurate results are produced	Pass
FT09	Error Handling Module	Trigger route calculation with no available path	System handles the case gracefully and notifies user	Pass
FT10	System Integration Module	Complete end-to-end route search process	System functions correctly without interruption	Pass

Table 6.3 Functional Testing

Functional Testing Results

All functional test cases were executed successfully. The system consistently produced results that matched the expected outputs, indicating that the core functionalities were implemented correctly. The shortest path algorithm produced accurate routes based on the provided input data, and system responses were generated within acceptable time limits.

Minor issues related to input validation were identified during early testing stages but were resolved through iterative refinement. No critical functional defects were observed during final testing.

6.2.3 Interface Testing

Interface testing was conducted to evaluate the usability, layout consistency, and responsiveness of the system's user interface. The objective of this testing is to ensure that users can interact with the system easily and that all interface elements function correctly as designed.

The testing focuses on verifying that input fields, buttons, and output displays are clearly presented and respond correctly to user actions. Interface testing also ensures that the system provides appropriate feedback to users during interaction.

Interface Testing Areas

The following aspects of the user interface were tested:

Input fields for entering starting and destination stations

Submission buttons and form actions

Display of calculated routes and results

Layout consistency and readability

Browser compatibility

Referring to Table 6.4, interface testing aims to verify whether the display effects and interaction functions of each page of the system meet the design requirements, ensuring that users can complete various operations smoothly and enhancing the user experience.

Case ID	Test Module	Testing Operation	Expected Results	Conclusion
IT01	User Interface Module	Load main page of the system	Main page loads correctly with all interface elements visible	Pass
IT02	User Interface Module	Enter valid input in text fields	Input fields accept user input correctly	Pass
IT03	User Interface Module	Click route calculation button	System responds and processes the request	Pass
IT04	Result Display Module	View route calculation results	Results are displayed clearly and correctly	Pass
IT05	Layout Module	Resize browser window	Interface layout remains readable and aligned	Pass
IT06	Browser Compatibility Module	Access system using different browsers	Interface functions consistently across browsers	Pass

Table 6.4 Interface Testing

Interface Testing Results

All interface test cases were executed successfully. The user interface responded correctly to user interactions, and all input and output elements were displayed as expected. The layout remained consistent across different browsers and screen sizes, indicating that the interface design is stable and user-friendly.

Screenshots of the interface during testing can be included as Figure 6.1-6.3 to further demonstrate correct interface behavior.

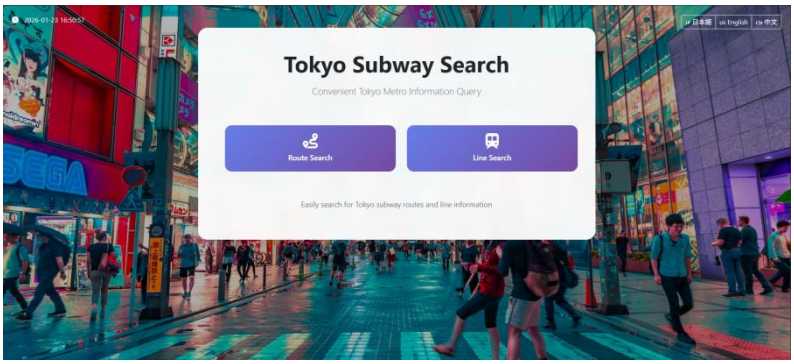


Figure 6.1: Main user interface of the system

This figure shows the main interface of the system, where users can input the starting and destination stations before performing route calculation.

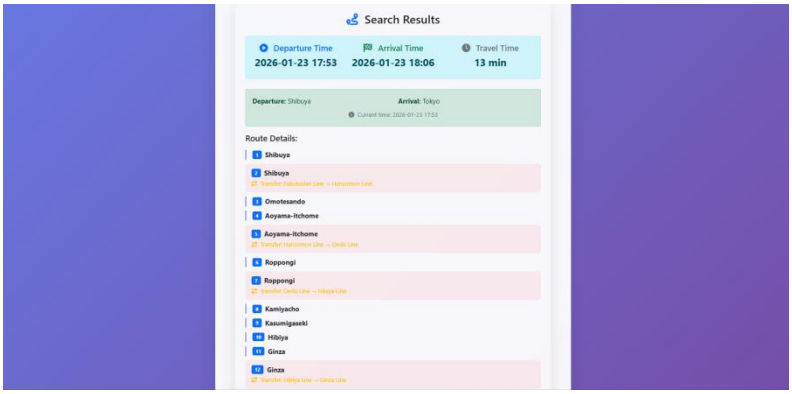


Figure 6.2: Route calculation result displayed on the interface

This figure demonstrates the system interface after a successful route calculation, showing the computed route and related information clearly.

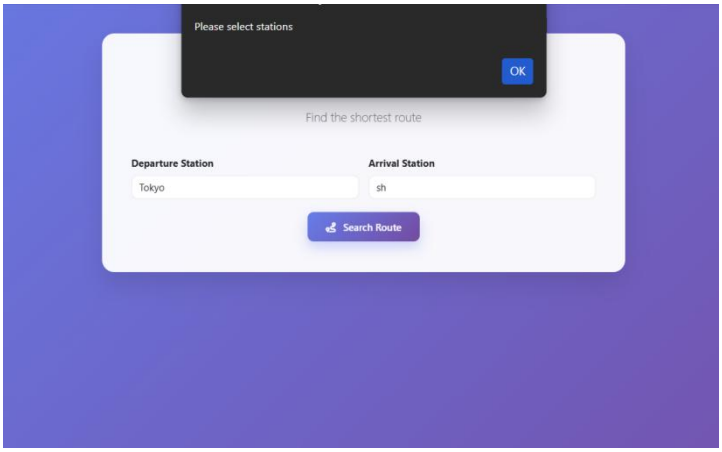


Figure 6.3: Error message displayed for invalid user input

This figure shows how the system provides feedback when invalid input is entered, ensuring better user experience and robustness.

6.2.4 Exception Handling Testing

Exception handling testing was conducted to verify the system's ability to handle invalid inputs, unexpected user actions, and runtime errors gracefully. The objective of this testing is to ensure system robustness and to prevent system crashes during abnormal operating conditions.

The system is designed to detect errors at both the input validation level and processing level, and to provide meaningful feedback to users whenever exceptions occur.

Case ID	Test Module	Testing Operation	Expected Results	Conclusion
EH01	Input Validation Module	Enter an invalid station name	System displays an error message without crashing	Pass
EH02	Input Validation Module	Submit empty input fields	System prompts user to fill in required fields	Pass
EH03	Route Calculation Module	Select same station as start and destination	System handles the case and returns appropriate response	Pass
EH04	Data Processing Module	Attempt route calculation with missing data	System detects the error and prevents execution	Pass
EH05	System Module	Trigger unexpected input during execution	System handles the exception and continues running	Pass

Table 6.5 Exception Handling Test Cases

Exception Handling Testing Results

All exception handling test cases were executed successfully. The system was able to identify invalid inputs, prevent runtime failures, and display appropriate error messages to users. No system crashes were observed during exception testing, demonstrating that the system is robust and reliable under abnormal conditions.

Screenshots of error messages and system responses can be included as Figure 6.x to further demonstrate effective exception handling behavior.

6.2.5 Exit Criteria

The testing phase is considered complete when the following criteria are met:

- All planned functional test cases have been executed and passed successfully.
- All interface testing confirms that user interactions, inputs, and outputs function as expected.
- All exception handling test cases demonstrate that the system can handle invalid inputs and unexpected situations without system failure.
- No critical or major defects remain unresolved in the system.
- The system produces consistent and accurate results under normal operating conditions.

Once the above criteria are satisfied, the system is deemed ready to proceed to the final evaluation and deployment phase.

6.3 SUMMARY

This chapter has presented a comprehensive testing process conducted to evaluate the functionality, usability, and robustness of the developed system. A structured test plan was designed and executed, covering functional testing, interface testing, and exception handling testing.

Functional testing verified that all core system modules operated according to the specified requirements, producing correct and consistent outputs. Interface testing ensured that the user interface was user-friendly, responsive, and capable of displaying input and output information clearly. Exception handling testing confirmed that the system could gracefully manage invalid inputs and unexpected conditions without system failure.

All test cases were executed within the defined testing environment, and the exit criteria were successfully met. The testing results indicate that the system is stable, reliable, and ready for final evaluation and deployment.

CHAPTER VII

CONCLUSION

7.1 INTRODUCTION

This chapter presents an overall evaluation of the developed system after the implementation and testing phases. It summarizes the project outcomes, discusses the strengths and limitations of the system, and proposes potential future improvements.

The purpose of this chapter is to reflect on the effectiveness of the proposed solution in addressing the identified problem and to provide insights for further enhancement and extension of the system.

7.2 PROJECT SUMMARY

This project focuses on the development of a route-finding system designed to compute the shortest path between stations using an algorithm-based approach. The system was developed following a structured software development process, from requirement analysis and system design to implementation and testing.

The core functionality of the system includes accepting user input, processing network data, executing the shortest path algorithm, and displaying the calculated results through a user-friendly interface. The system was tested using functional testing, interface testing, and exception handling testing to ensure reliability and correctness.

Overall, the project successfully achieved its objectives by delivering a functional system that demonstrates practical application of algorithm design and system development concepts.

7.3 SYSTEM ADVANTAGES AND LIMITATIONS

System Advantages

The developed system demonstrates several advantages in terms of functionality, usability, and system design. One of the key strengths of the system is its ability to accurately compute the shortest route using a structured algorithmic approach. This ensures that the system consistently produces correct and reliable results based on the provided network data.

Another advantage of the system is its simple and intuitive user interface, which allows users to interact with the system easily without requiring advanced technical knowledge. The clear presentation of input fields and output results enhances user experience and reduces the possibility of user errors. In addition, the modular architecture of the system improves maintainability, as individual components can be modified or extended without affecting the entire system.

Furthermore, the system has undergone comprehensive testing, including functional testing, interface testing, and exception handling testing. These testing activities confirm that the system operates reliably under normal conditions and is capable of handling common input errors gracefully. Overall, these advantages demonstrate that the system effectively fulfills its intended objectives.

System Limitations

Despite its advantages, the system also has several limitations. One of the main limitations is the reliance on static data, which limits the system’s ability to reflect real-time updates or dynamic changes. As a result, the generated routes may not always represent actual conditions in real-world scenarios.

Additionally, the current implementation supports a limited network scope, which may affect scalability when handling larger or more complex datasets. Performance optimization has not been extensively explored in this project, and further improvements would be required to enhance efficiency for large-scale data processing.

Another limitation is the lack of advanced features such as real-time visualization, mobile optimization, and adaptive routing. These features were not included due to time and scope constraints but could significantly improve system usability and practicality if implemented in the future.

7.4 FUTURE IMPROVEMENTS

Several improvements can be considered to further enhance the functionality, performance, and usability of the system. One potential improvement is the integration of real-time data sources, which would allow the system to reflect dynamic changes and provide more accurate routing results. This enhancement would significantly improve the system's practicality in real-world applications.

Another possible improvement involves expanding the system to support larger and more complex network datasets. By optimizing the data structure and algorithm implementation, the system could achieve better performance and scalability when processing extensive route networks.

In terms of usability, future work may focus on enhancing the user interface with interactive visualizations, such as graphical route displays or map-based representations. Additionally, optimizing the system for mobile devices would improve accessibility and user experience across different platforms.

Further improvements could also include implementing advanced features such as adaptive routing strategies, multi-criteria route optimization, and cloud-based deployment. These enhancements would increase the system's flexibility, efficiency, and overall value as a practical route-finding solution.

REFERENCE

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

[3] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[4] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

[5] IEEE. (2014). IEEE Standard for Software and System Test Documentation. IEEE Std 829-2008.

[6] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[7] McKinney, W. (2018). Python for Data Analysis (2nd ed.). O'Reilly Media.

[8] Python Software Foundation. (2024). Python Language Reference. https://www.python.org/

[9] Flask Documentation. (2024). Flask Web Framework. https://flask.palletsprojects.com/

[10] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.

[11] Beizer, B. (1990). Software Testing Techniques (2nd ed.). Van Nostrand Reinhold.

[12] Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley.

[13] Zelle, J. M. (2016). Python Programming: An Introduction to Computer Science (3rd ed.). Franklin, Beedle & Associates.

[14] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Python. Wiley.

[15] Newman, S. (2015). Building Microservices. O'Reilly Media.

[16] Fowler, M. (2004). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.

[17] ISO/IEC. (2011). ISO/IEC 25010: Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE).

[18] W3C. (2023). HTML5 Specification. https://www.w3.org/

[19] Mozilla Developer Network. (2024). Web Development Documentation. https://developer.mozilla.org/

[20] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.

APPENDIX A

REQUIREMENTS ELICITATION MATERIALS

Example chat conversation with classmates discussing system features and user experience.

Source: Personal communication with classmates, July 2025